CORA: Learning What to Change Through Executable Reasoning and Failure-Grounded Capability Construction

Technical report, November 2026. Not peer reviewed.

Chimdi Walter Ndubuisi* and Toni Kazic
University of Missouri *cnptp@missouri.edu

Abstract

When a reasoning system cannot solve a problem, more search may not help: its operations may be unable to express the solution. CORA (Constructive Operator Reasoning Architecture) treats failure as evidence. When search fails within a set budget, CORA uses the task's examples to describe what a missing step in a program must do (a typed hole). It tests possible operations from a predefined set and keeps one only if it passes checks. In a pre-registered experiment, we created 95 tasks, each missing one of 211 supplied operations. CORA solved 64 tasks in all three seeded runs. Using the examples directly solved 48, ordinary search solved 38, and using another task's information solved 16. Ordinary search used 3.2 times the CPU charged to construction. However, none of the 211 operations met the requirements of 425 failed benchmark tasks, so we developed a method for building operations that change individual objects. We then tested whether saved operations could help on other problems. The test covered 624 outputs from three ARC-style benchmarks, kept out of this study's development but used in earlier studies. Ordinary search produced 118 correct answers, reusing saved operations 227, and adding object-level construction 343; 286 of the 343 came from one-dimensional tasks, and on the 315 two-dimensional outputs the three methods produced 30, 11 and 57. We also rebuilt the approach for logic problems: it recovered 45 of 217 held-back logic functions, compared with 12 when the failure information was shuffled. On 84 generated tasks, a fixed rule for choosing what to try next solved 55, a preset order 29 and a random order 37; an advantage over the best order chosen after testing (44) was not established. CORA's Kaggle entry, which checks answers but does not build operations, scored 0.83% on 120 hidden ARC-AGI-2 tasks. These experiments show that a task's examples can help identify and reuse useful operations within a supplied set. Learning which change to make, and building operations beyond that set, remain open challenges.

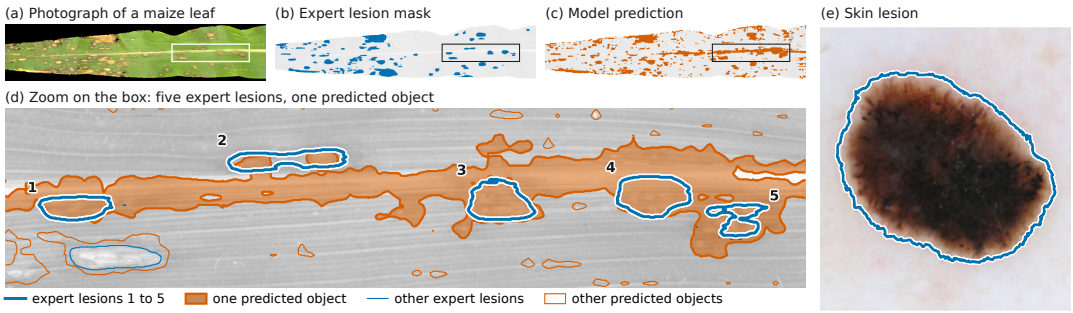


Figure 1: Lesions show why the representation comes first. (a) Maize leaf, (b) expert lesion mask, (c) model prediction; (d) one predicted object (orange) merges five expert lesions (blue). (e) A skin-cancer lesion and its expert boundary (ISIC Archive image ISIC_0000013 and its ISIC 2018 mask, both CC0; the ISIC record documents a histopathology-confirmed invasive melanoma, which the image itself does not establish). Motivation only, not CORA data and not a diagnosis: (a) to (d) show a validation leaf of our unpublished lesion study.

1 Introduction: why reasoning needs to change its own capabilities

A reasoning system explains observations with the operations it has: it searches over programs built from them and keeps one that reproduces what it has seen. When it fails, the cause can differ. The explanation

The extended version, with Appendices A to I, is in the public repository (Code and data).

may exist but be hard to find, the observations may be described with the wrong objects, or an operation the explanation needs may be missing; each cause calls for a different change. Our broader question, which grew out of our research discussions, is whether a system can learn from a few observations, notice when its rules fall short, work out what must change, change it, and carry the same way of reasoning to a new domain.

Lesions show why the representation comes first (Figure 1). For object-level reasoning about lesions, the chosen boundaries constrain which rules can work. On the maize leaf, one predicted object merges five expert lesions; rules using that object inherit the merge unless the representation changes. Our unpublished leaf-lesion work, which does not use CORA, pointed the same way: what had to change was seeding, where objects start, not geometry [49, 50]. When a separate study ran CORA's image pipeline on our leaves, with every decision frozen before any evaluation mask was read, a segmenter trained on cell images found almost none of the lesions on 16 evaluation leaves, CORA's checks refused every edit, and a plain search's mostly helpful edits (103 of 120) left F1, the lesion-detection score, near zero: the tested local patches could not rescue a wrong representation.

Searching longer and changing the language are different responses. Refinement loops, the defining theme of ARC Prize 2025 [18], improve a program or a network within a supplied language. Longer search reaches behaviour that deeper compositions express; behaviour outside the language's expressible set needs a new operation, and behaviour reachable only through too deep a composition can benefit from one: *capability construction*.

When reasoning fails, the failure is evidence; CORA uses that evidence in two distinct ways. In the ARC-style construction experiments, a failed bounded search triggers construction, and the task's demonstrations specify what the missing operation must do. In the logic-function experiments, the failure description itself carries information used to choose the revision: shuffling it removed most of the gain. In the second selection test, the deciding fact is a property of the demonstrations stored in CORA's failure record, whether every demonstration keeps its grid size; it does not come from the failed search. In every case a proposed change is checked against the observations before it is kept.

CORA (Constructive Operator Reasoning Architecture) turns this into one procedure: observe, represent the observations as objects and relations, search for an executable program, and answer only with a checked program; when bounded search fails, work out a *typed hole*, the input and output pairs a missing operation must produce, test candidates from a declared space against it, and keep one only if it passes explicit admission checks (Figure 3 follows one recorded task). Three requirements organise the evidence: *capability availability* (can the language express the explanation?), *capability discovery* (can the system find it within its budget?) and *capability selection* (can evidence decide which representation, operation or strategy to use?). Construction expanded budgeted reach and improved discovery inside a supplied space; selection is the open frontier. The same procedure was rebuilt for other domains through *adapters*, which translate a domain's observations into the loop's entities and operations: the procedure transfers, rebuilt per domain; a domain's own evidence beat shuffled evidence in two of five new domains, and learned grammar transfer across distant domains is not shown. The box separates what was scored from these research studies.

Scored entries. Kaggle submission 57902131 runs CORA alone, on CPUs, with no neural model. It submits a program-derived answer only after checking it, and otherwise a copy of the query input, its fallback for an internal abstention (on the public evaluation set its only 3 correct outputs were its 3 answers). Kaggle public-leaderboard score on 120 hidden ARC-AGI-2 tasks: **0.83%**. Submission 57867631, the neural comparison entry, runs our re-implementation of NVARC's published method on NVARC's checkpoint [65], with a checked CORA second attempt: **24.03%**, 3.6 points below NVARC's own notebook, which we reran as a reference (submission 57160429, not ours: 27.64%). CORA's second attempt gained 0 and lost 0 answers on the public evaluation set, and its effect on the hidden tasks is not measured. The full research paper [51] holds every study, control and cost.

Scope. The scored entries run search and checking only (an earlier entry with failure-driven construction, 57354496, scored 0.00%); this paper's construction, reuse, the learned order, adapters and biology code ran in separate studies. We supplied the constructor's space of operations, and fixed rules decide when to construct. The generated tasks test the mechanism; they are not ARC accuracy. Held-back halves were held out from each study's development only: every task had been scored once on 2 November 2026 by our submitted entries and once by a parallel study's evaluation. Boolean gene-network observations are simulated from published models, so those results are reconstruction, not discovery; the single-cell, microscopy and spatial studies use measured data and support scoped diagnoses or adapter tests.

Contributions. The ingredients have precedents, and Appendix H states each prior system and the exact difference. Our contributions are implemented designs, controlled comparisons and identified limits:

1. *What a build reads decides what gets built.* Bias shift and predicate invention, also from failures, add to a learner's vocabulary, and Popper and Neo learn failure-based constraints [20, 21, 24, 48, 67]; counterexample-guided synthesis refines candidates within a fixed grammar, abstraction refinement refines an abstraction from a spurious counterexample, and witness functions derive what a missing piece must do within a fixed language [19, 56, 64]. Our pre-registered controls change only the evidence a build reads: over the same 211 operations, a task's own typed holes solved 64 of 95 generated tasks, the operations tried blind 32 (by three-repeat majority) and another task's holes 16; in logic, 45 of 217 held functions against 12 with the failure description shuffled.
2. *Availability and discovery, measured on the same tasks.* DreamCoder, LILO and Stitch learn abstractions from discovered programs [9, 23, 27]. Of 624 previously used held outputs, the failure-kept library got 227 (216 on 1D-ARC; known beforehand), same-size compression of solved programs 28, random libraries 6, with unequal build CPU (not a general method comparison).
3. *When choosing the change can pay*, from two pre-registered revisions of a selection test, beside learned search guidance, which in DeepCoder and CrossBeam beat its baselines, and metareasoning [6, 58, 61].
4. *One loop design, rebuilt by adapters* for five new domains, each frozen before its held run, every held result reported with its scope, a null and a failed test among them; FlashFill serves one domain [28].

Sections 2 and 3 give the mechanism and its limits in principle, Sections 4 to 7 test construction, reuse, selection and transfer, and Sections 8 and 9 turn the failures into the next experiments.

2 From failure to capability construction

Why ARC-AGI-2. A few input-output examples constrain a hidden transformation, and candidate executable programs can be checked against them without a human judge [11, 17]; each failed program leaves a record of what went wrong, two attempts are allowed per query output, and the scoring tasks are hidden [18, 35]. The Boolean gene-network studies use simulated examples, which likewise constrain a hidden rule.

Terms. A *task* gives *demonstrations*, pairs of input and output grids, and *queries*, test inputs whose *query outputs* must be produced; a system *answers* a query output by giving a grid, *correctly* if an attempt matches exactly, and *solves* a task by answering its query outputs correctly. A *program* is a short sequence of typed operations; a *language* is the operations and the rules for combining them; the *starting language* is the language before any operation is added, and *static search* searches it with no new operation. In the scored entry, *checking* runs a program on every demonstration and rebuilds it without each one; *abstention* is declining to answer. *Held out* means kept away from a study's development; *exposed* means used before in this project, so held-out exposed tasks are never unseen. *Pre-registered* means frozen before scoring, and *audited*

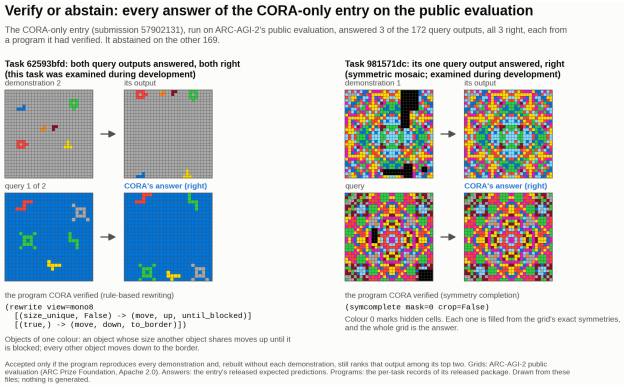


Figure 2: An actual executable ARC explanation: every answer of the CORA-only entry on the public evaluation set, drawn from the task file and the entry's recorded programs (an actual CORA run). It answered 3 of 172 query outputs, all 3 correct, each from a program that reproduces every demonstration and, rebuilt without each demonstration, still ranks that demonstration's output among its top two; it abstained on the rest. Both tasks were inspected during development, so this shows how CORA explains a task, not an unseen accuracy.

means checked after scoring, from saved records, by a separate review within this project, not by external replication. Each study states its *counted-work* unit (candidate applications, called *counted operations*, or primitive executions): machine-independent, and not interchangeable across studies.

The scored entry: answer only with a checked program. CORA's grid adapter reads each grid as objects, connected regions cut in one of several ways (*views*), with colour, size, shape, position and relations. The CORA-only entry runs two CORA solvers in turn. The main one, CORA-CAST, builds programs with symmetry completion, guarded rewriting (short first-match lists of "if an object has this property, do this action") and typed search pruned by sound contracts. It answers only if a program reproduces every demonstration and, rebuilt with each demonstration left out, still ranks that demonstration's output among its top two, and never from a single demonstration; otherwise it abstains, within 45 CPU seconds and 90 wall seconds per task. A second reviewed CORA solver from the companion study, a parallel line of our experiments, adds stricter leave-one-out checks. Run once, unchanged and untuned, the entry's code was correct on 81.5% to 95.8% of the query outputs it answered on three related benchmarks, while answering only 11% to 50% of them, and with another task's demonstrations precision fell to at most 2.4% (exposed; pre-registered): on exposed sets coverage looks like the main limit, and how the official 0.83% splits into precision and coverage on the hidden tasks is not observed.

Construction: from a failed search to a kept operation. The scored entry stops at checking. When search fails, a separate research prototype tries to construct the missing operation. A program sketch puts the hole alone, or before or after one invertible starting-language operation; undoing that operation (if any) on the demonstrations gives the exact input and output pairs the missing piece must produce: the failure triggers the step, and the demonstrations supply its content. In the generated-task study all 211 declared whole-grid operations are checked on these pairs, in about 0.02 CPU-seconds, and only matches enter a search; the held-half study's constructor is object-level instead (Section 5). A candidate is kept only if a bounded search in the starting language does not reproduce it, it fits the task, it breaks no solved task and it fits another unsolved task; kept pieces join the language. This is selection within a known space, not open invention: we supplied the operations and the diagnosis follows fixed rules. Learning enters only through kept operations and a learned search order, the direction the organisers' 2024 report expected for guiding program search [16].

A worked example. Figure 3 follows one of the 95 generated tasks. Its four demonstrations each shrink a grid, and the task was generated using an operation omitted from the starting bank. Search still found no fit at four times the budget; with diagnosis unresolved, the controller chose construction. The top sketch is the hole alone, so the hole equals the raw demonstrations; 4 of the 211 passed it, and the one sent to the checks, a row-and-column selection, passed them all. Seeds 1 and 2 admitted that selection and seed 0 a sibling variant; each solved the task, and saved results show five later tasks solved using the retained variant.

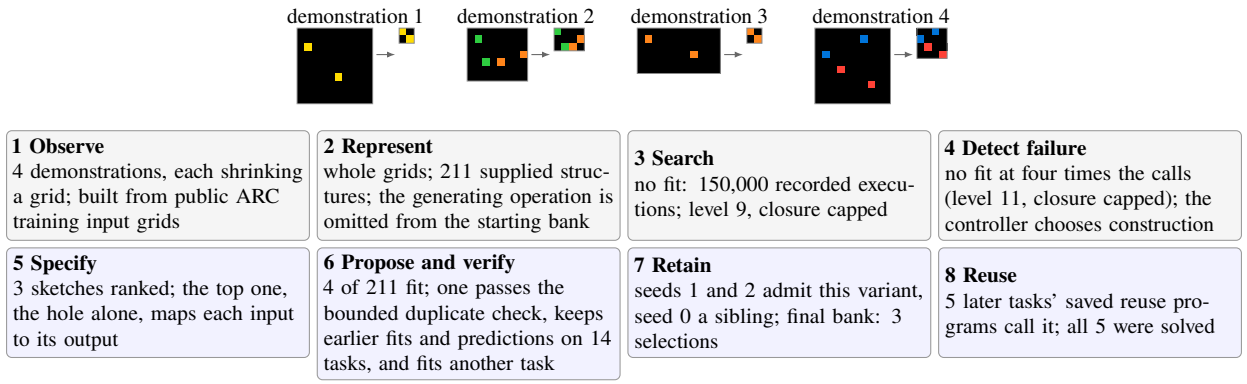


Figure 3: CORA's mechanism on one generated task of the construction study, step by step from its records (an actual run of the research prototype; the scored entry stops after checking). Top: the four demonstrations (the query is not drawn); bottom: the recorded sequence. Seed 0 admitted a sibling variant, which the committed language does not hold. The top sketch is the hole alone, so here the typed hole equals the raw demonstrations, which also solved the task; the same 211 operations tried blind solved it in two of three repeats; another task's holes and static search failed (no fit after 379 CPU-seconds and 22.9 million recorded executions, level 14, closure capped). Every search had a cap, so this does not show the program inexpressible. The five reuse outcomes are the original run's saved results, not recomputed and not proof that the operation was necessary. For a task where the task's own typed hole succeeded and the other arms failed, see Section 4 (our key td0c3d452).

3 Theoretical limits of search, construction and selection

The worked example raises three questions. Can demonstrations identify the intended program? Not necessarily: several programs can reproduce the same observations and differ elsewhere. Can better selection make up for an unavailable operation? No: a selector cannot choose an answer absent from its candidates. Can enlarging the language create new problems? Yes: added fitting programs can increase ambiguity under some acceptance rules. Four elementary conditional results, proved in Appendix B under their stated assumptions, make these answers exact; Table 4 separates empirical illustrations from the untested prediction, the results organise the measurements rather than predict them, and no priority is claimed. Let D be the demonstrations, x^* a query, $\mathcal{L}$ a language, $V_{\mathcal{L}}(D)$ the programs of $\mathcal{L}$ that fit D (reproduce every demonstration), and $A(D, x^*)$ their predictions at x^*.

Proposition 1 (Many explanations)**.** *If some program of $\mathcal{L}$ fits D, $\mathcal{L}$ can return any allowed output on an exact match to x^* and otherwise run any program of $\mathcal{L}$, and x^* differs from every demonstration input, then every allowed output is predicted at x^* by some program in $V_{\mathcal{L}}(D)$ [46].*

Proposition 2 (Build before you rank)**.** *If attempts are drawn from the retained predictions within budget b, no selection rule, however learned, raises accuracy above the share of tasks whose answer is retained, and that share cannot exceed the share of tasks whose answer some program of $\mathcal{L}$ that fits D predicts at x^*.*

Proposition 3 (Bigger language, more ambiguity)**.** *Suppose an answer is given only when every fitting program returns the same output at x^*, a program that returns nothing counting as disagreement, and $\mathcal{L} \subseteq \mathcal{L}'$. On every task that $\mathcal{L}$ already fits, growing to $\mathcal{L}'$ never adds or changes an answer; it turns an answer into an abstention exactly when $\mathcal{L}'$ adds a fitting program that does not return that output at x^*.*

Proposition 4 (A missing input blocks exact recovery)**.** *Say that a rule depends on an input v when some two inputs that differ only in v receive different values. If a rule depends on v and every candidate gives equal values to any two inputs that differ only in v, no candidate equals the rule, however good the search; and for queries fixed in advance, or chosen by a fixed deterministic policy from earlier binary responses, and with no target-dependent side information, m binary responses can single out at most 2^m rules [1, 4].*

What they mean for CORA. Passing the examples shows consistency, not the intended rule (Proposition 1), which is why CORA's answer rule also rebuilds each program without each demonstration: of 4,832 constructed rules proposed by two or more demonstrations and fitting all, 562 were wrong on the query (57.9% on ConceptARC; post hoc, development data). A selector cannot return what the language cannot

express or search cannot reach (Proposition 2): on the 41 query outputs that only the neural comparison entry answered correctly, ten times CORA's search budget found no fitting program, so no ranking of CORA's candidates could have answered them. Growth has a cost under all-agree acceptance (Proposition 3, not yet tested). A missing input blocks every candidate (Proposition 4), the gene-network form of a missing operation (Figure 5b), and the bound is why choosing observations matters. These mathematical statements, under their assumptions, say when construction and selection are necessary; the experiments below test whether CORA's implementation uses that opportunity within a budget, and the propositions prove no empirical advantage.

4 Does construction improve capability discovery?

Question and design. If the language lacks an operation, can task-specific evidence guide its construction better than search or blind trial? A frozen study built 95 tasks from public ARC training input grids in three worlds whose languages lack 11 of the 211 operations (scalings, crops, symmetries, colour swap; pre-registered; audited; three seeded repeats). Each control changes one ingredient (Table 13, Appendix I).

Result. Construction from each task's own typed holes recovered 64 in all three seeded repeats (60 by admission, 4 by a fitting fallback), against 38 for static search, which used 8.47 measured CPU-hours in one run, while construction was charged 2.61 CPU-hours for its three repeats under the protocol's complete-cost rule, which charges each cached admission check at its first computation (measured growth about 0.31 CPU-hours plus 902 diagnosis CPU-seconds): 31 tasks gained, 5 lost, a difference of 27.4 points (95% interval over eight family clusters, +5.1 to +40.6). The raw demonstrations as the hole recovered 48 (16 gains for the typed holes, 0 losses), another task's holes 16, all in the one world whose removed operations compose (0 of 59 in the other two), and the same 211 operations tried blind 32 (3 in all repeats, 74 in any; Figure 4a).

Principal finding. With the supplied candidate operations and the checks fixed, holes from each task's own demonstrations recovered more generated tasks than tested alternatives (64 against 48 for raw demonstrations and 16 for another task's holes): within these generated tasks, the information read caused the gain.

Depth and limits. Post hoc, the 31 tasks that only construction solved split into the two requirements (Figure 4b): 13, the worked example among them, have no equivalent starting-language program at any depth searched (not proof that none exists deeper), 17 have one 1 to 4 cost levels deeper, an estimated 1 to 135 times static search's budget (median about 5), and 1 was a static-search process failure. On one of the 17 (our key td0c3d452), the winning recorded sketch first mirrors the inputs, then fits the colour exchange, in all three repeats; raw demonstrations, a budgeted blind trial and another task's holes failed (saved outcomes; an illustration, not an accuracy result). Leaving out either of two worlds gives intervals that include zero, and every removed operation belongs to the space by design: selection within a known space, not open invention. Is a constructed operation useful beyond its own task? Within this study, operations reused on other families' tasks did worse than static search (23 against 28 of 60; Holm $p = 0.47$).

5 Can constructed operations be reused?

Question and design. Learning a useful operation differs from learning one that generalises. Object-level operations built on the source halves of ConceptARC [47], Mini-ARC [34] and 1D-ARC [69] ran once on the held halves, 624 query outputs of 464 tasks, every one scored once before by the submitted entries (exposed; pre-registered; audited). The held halves come from the same task families as the source halves, and the object-level constructor fits one or two first-match rules, each a supplied predicate (among them 24 selection features) with a supplied object edit, over 12 views (Appendix I).

Result. Reuse, the pre-registered primary, was correct on 227 against 118 for static search in the starting language with the same CPU allowance (+109, 95% interval 7 to 204), all of the gain on 1D-ARC; on the two two-dimensional benchmarks reuse lost (11 against 30 of 315). Object-level construction from each task's own demonstrations, where reuse failed (a pre-registered secondary), was correct on 343 of 624 against the same 118 (it answered 377; interval for the difference 150 to 298), and on 57 against 30 of 315 in two dimensions.

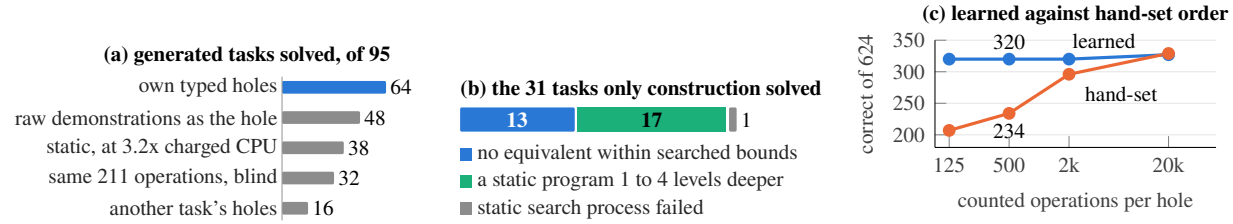


Figure 4: How well construction and search guidance work: the central controlled evidence, from actual runs of the research prototype (Figure 3 shows the mechanism). (a) Generated tasks solved, of 95, each lacking one operation (pre-registered; audited), counted in at least two of three seeded repeats; static search used 3.2 times the CPU charged to construction (measured against charged); the blind trial of the same 211 operations solved 3 in all three repeats, and the other four counts are the same in all three. (b) Post hoc: the 31 tasks that only construction solved; "no equivalent within searched bounds" is not proof that none exists deeper. Every removed operation belongs to the space by design (for 7 of the 11 families the space was fixed before the operation was chosen, and the other 4 operations predate it), and the blind control searches the same space: selection within a known space, not open invention. (c) Correct query outputs of 624 held, exposed outputs at four caps on counted operations per hole; the hand-set order first reaches 320 at 20,000, 40 times the test cap of 500 (a sampled per-hole cap, not total compute and not a proven minimum).

Static search given the same expanded operation bank has not been run. A library of whole programs beat the starting language (37 against 6) but lost to static search given the library's whole cost (37 against 56 of 624); that tests our library, not DreamCoder or LILO [23, 27]. On the 120 public ARC-AGI-2 evaluation tasks, construction correctly answered 2 of 172 query outputs, both on the left task of Figure 2, which the CORA-only entry already answers; every other condition solved none (95% interval for the difference from static search, 0 to 6 outputs): a descriptive, exposed example, not evidence that the operation was necessary; reach is the limit there.

What it means. Reuse can help only where operations were kept: it gained on 1D-ARC (19 kept) and lost in two dimensions (2 kept, none on Mini-ARC). When should the system reuse an operation, search longer, change its representation or construct another one?

6 When does evidence help select a revision?

Two activities must be kept apart: *search guidance* decides where to search within an available operation space, and *revision selection* decides which kind of change to attempt. The left task of Figure 2 shows why selection is hard: five earlier versions of our system failed on it, with no fitting program, for four different reasons, each needing a different change (Appendix I; inspected during development: an illustration).

Search guidance helps when the budget binds (exposed; pre-registered; audited). A small guide trained on earlier successes and failures replaced the constructor's hand-set order (Figure 4c): at 500 counted operations per hole it was correct on 320 of the 624 held query outputs against 234 (search CPU 115 against 106 seconds); at a per-hole cap 40 times larger, one of four sampled caps and not a measure of total compute, the counts were close (327 against 329; inconclusive). A frequency table recovered 318 and a linear model 323, against 320 for the learned guide, so in this study the learned guide shows no advantage over simpler rankers. A later guide reading each task's grid pairs and the constructor's own analysis of its search blocks, frozen and reviewed before any held task existed, solved 1,083 of 1,128 held tasks generated from ARC-AGI-2 public training input grids at 125 counted operations each, against 953 for the hand-set order, 881 for a frequency table and 1,018 for a linear model (pre-registered; audited): search efficiency at a binding budget, not new reach; a guide reading only the constructor's own block features solved 1,076, not significantly fewer.

Revision selection (generated; pre-registered; scored once; audited). Two revisions of one test varied only the order of the same changes under the same resource caps and work meter; an *oracle* router is told each task's injected cause. In revision 1, typed-hole construction rejects an inapplicable hole after about 900 calls, so trying it first costs almost nothing, and the best fixed order matched the oracle (30 of 50 each): choosing added nothing. Revision 2 added a change that cannot check cheaply whether it applies, a search over alternative object views. In a pre-registered test on 84 controlled generated tasks (transformation tasks

built from ARC training input grids, not ARC puzzles), each with one injected cause of failure, a fixed router that reads one fact from CORA's failure record, whether every demonstration keeps its grid size, solved 55 tasks at the pre-declared binding budget of 3,000,000 primitive executions, against 29 for the static full bank in its pre-registered order (26 tasks against 0; exact McNemar p = 3.0e-8; 95% interval over resampled task families, +11.8 to +55.2 points). The whole gain came from the 28 wrong-object-view tasks, and at every budget the router solved exactly the oracle's tasks.

Its limits. Against the best fixed order chosen after seeing the held results (construction, then the view search, then deeper search: 44 of 84), the router's advantage is not established: 11 tasks against 0, with an interval from 0.0 to +26.1 points. The size fact separates the causes because of how the tasks were generated, and it comes from the demonstrations rather than from the failed search. The pre-registered static order scored below a random order on the held mix (29 against 37), and two of the three held causes reuse revision 1's held families with fresh input grids. A learned Boolean selection study's pre-declared gate stopped its held study, so no held test ran (Appendix D).

What it means. Where a change cannot cheaply check whether it applies, no fixed order of the bank reached the oracle (best 44 against 55 of 84), and the pre-registered router did, with no established advantage over that best order: choosing the next revision can pay when demonstration evidence separates the injected causes and an unsuitable change is costly. This is not revision selection on natural ARC tasks, where all CORA programs together solve 2 of the 120 public tasks, too few to compare methods. What carries over to other domains?

7 Does the reasoning procedure work beyond ARC?

What transfers. Transferability here means that the reasoning pipeline carries over (build an executable rule, check it, diagnose failures, revise or ask for evidence, abstain when unsure), not that puzzle rules apply to genes. The loop design is reimplemented through domain-specific adapters, using supplied building blocks and checks. An observation is a grid pair or a simulated network state paired with one gene's next value. Entities are objects or genes, programs are grid transformations or gene update rules, and inferred inputs remain candidates. Checks compare outputs. Spatial adjacency is not regulation: the correspondence holds only through an adapter that declares its meaning. Evidence-guided extension within supplied spaces helped in some tests: reading the domain's own evidence beat a shuffled or borrowed one on generated grid tasks, in logic and in signalling, and is not established in spreadsheets, chemistry or ARC-AGI-3 (Table 12); on measured imaging, the grammar built from failures was empty and no robust benefit emerged. Learned content transfers within 1D-ARC (Section 5) but not yet between distant domains: an operator order counted on ARC training tasks did not help gene rules (177 against 194 for a hand-set simplicity order). The scored CORA-only code ran unchanged on 44 published gene-network models but recovered no rule, so the gene results come from a rule-reconstruction procedure written for biology, which reuses CORA's verify-or-abstain acceptance design but none of the entries' code.

Controlled reconstruction: logic functions (held; pre-registered; audited). Each new-domain test rebuilt the loop through an adapter (building blocks, a checker and a failure description; each study's public adapter card lists what was reused). On 217 held real benchmark logic functions (the Espresso example files, which include the MCNC benchmarks; 6 to 12 inputs), recovered exactly from a random 25% of each truth table, the loop recovered 45, against 7 for the base language of two-input gates and 12 for the same loop with a shuffled failure description (34 to 1 on discordant functions; file-clustered 95% interval for the difference 0.075 to 0.241); CART, a descriptive reference, recovered 39. The shuffled control keeps every program the base language finds, so it tests whether the failure description itself carries the gain. The gain spread over 13 of the 35 files, 153 of the 217 were recovered by none of the six methods, and every recovery reconstructs a known function. Choosing three quarters of the probes where the surviving candidate programs disagreed most raised exact recoveries from 45 to 63 of 217 at the same probe budget.

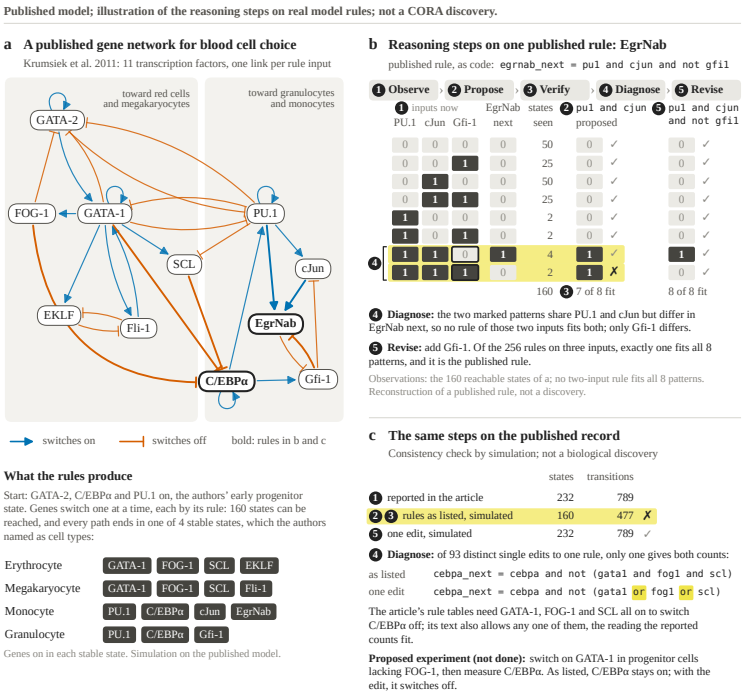


Figure 5: The reasoning procedure on a real rule of a published blood-cell gene network: a reconstruction from a published model, not a CORA run and not a discovery. (a) The 11 transcription factors of Krumsiek et al. [38] and their update rules, which from the progenitor state end in four stable states, the blood cell types. (b) A simpler rule is insufficient: the best two-input rule for EgrNab fails on one observed pattern; adding Gfi-1 to the illustrated candidate inputs leaves one of 256 three-input rules, the published one. (c) One edit to a listed rule matches the article's state counts (a consistency check by simulation), and a proposed experiment, not done, would separate the two readings. CC BY article.

Biological reconstruction: published gene networks (simulated; reused cohort). A published gene-network model gives each on-or-off gene an update rule over input genes, its *regulators* in that model; an observation pairs a simulated network state with that gene's next value, and exact recovery means a rule that agrees with the published one on every possible input state. On 595 rules from 75 models in 50 publication lineages, preferring candidate regulators inferred from each rule's ten simulated observations (kept apart from the documented ones), the biology code recovered 101 rules exactly, against 66 for each of three controls (two with no regulator preference, one with shuffled regulators), in a replication frozen before its single run; all 101 are one-input rules, more wrong rules were accepted (124 against 112), no classical support-inference method ran, and the models had been parsed before. Twenty observations chosen where fitting rules disagree gave 320 exact rules against 291 for random ones, but fewer correct held inputs (1,586 against 1,747 of 2,380). Conditional construction, which builds a rule over more inputs from two simpler rules, recovered 433 rules exactly at 40 observations, against 321 for the complete two-input rule language; all 112 additional rules have three or more inputs, so this shows executable coverage from a larger supplied language, the biology counterpart of building a missing operation, but a classical learner given the same observations also reached 433, and the run does not test whether a failure triggered the gain. Figure 5 shows the reasoning on one published rule; CORA's biology reaches computational validity only (Appendix E).

Other computational domains (held; pre-registered; audited). The other held tests are in Table 12 (Appendix I), with their controls and limits. Spreadsheets were a null (22 against 17 and 17 of 53). In signalling, 863 of 1,000 held conditions were predicted exactly against 805 with a shuffled description; 44 of 58 gains came from mek12 to erk12 (one of six edges); elsewhere 2.0 points, not significant. Tasks were synthetic; static cannot express the target; new draws repeat development; the weaker cross-time shuffle preserves readout and direction. In chemistry, the loop gave exact products for 76 of 200 held substrates, against 75 for nearest neighbour; the ARC-AGI-3 probe did not pass its pre-registered test.

8 What CORA cannot yet do

The failures fall into four kinds (Table 1), each stated as the measured result, what it taught us and the experiment it motivates: the thesis applied to the research itself. *Insufficient operation space.* On the public evaluation set (post hoc; exposed; not an official score), our neural comparison entry, with a contaminated

kind of failure	what we measured	what it taught us	what to change next
insufficient operation space	no typed hole of 425 failed source-half tasks matched one of the 211 operations; on 41 query outputs that only the neural entry answered correctly, ten times the budget found no program (post hoc, exposed)	the tested pieces did not match these holes	object-level construction, correct on 343 of 624 held query outputs; actions that draw on the background, edit object parts and assemble grids
inadequate representation	on 16 of our maize leaves a cell-image segmenter scored F1 near zero and CORA abstained on every leaf	the tested local edits did not rescue this representation	detect a collapsed baseline and change the representation first
ambiguous evidence	562 of 4,832 constructed rules that fit every demonstration were wrong on the query output (post hoc, development data)	fitting the demonstrations shows consistency, not the intended rule	choose observations where candidates disagree (45 to 63 of 217 in logic)
unreliable acceptance	a pre-registered calibrated acceptance rule, tested once on 100 held-out PUMA melanoma regions [60], accepted 605 edits, 430 harmful, lowering mean per-image F1 by 0.0020 (95% interval 0.0014 to 0.0027); the strict rule made no edit	it tested an acceptance rule, not learning from failure: its harm limit of 0.60, relaxed from 0.20 on the same calibration organs, did not hold on a new source (0.711)	tie any harm limit to the target's baseline accuracy, fixed before looking; abstain by default on a new source

Table 1: Four kinds of failure: what we measured, what it taught us, and what to change next; more rows are in Table 5.

measured limit (where it is measured)	next advance
a supplied operation space (0 of 425 failed tasks matched)	operations beyond the declared library, for example a learned proposer for typed holes whose proposals CORA checks
uncertain revision selection (no shown gain over best fixed order; 2 of 120 natural tasks)	learn when to change the representation, construct or search longer on natural tasks; a real-puzzle protocol has passed review as a draft
limited reuse (gain only on 1D-ARC)	learn when a kept operation applies; check it on another family first
ambiguity (562 of 4,832 fitting rules wrong)	choose informative observations (45 to 63 of 217 in logic)
noisy data (430 of 605 edits harmful)	calibrate acceptance on the target domain before accepting edits
no validated biological discovery	frozen predictions tested on independent measurements and interventions

Table 2: From measured limits to the next advances, ordered from problems already measured to harder, unsolved ones; none is claimed as solved.

checkpoint, was correct on 43 of the 172 query outputs in rehearsal and the CORA-only entry on 3; of the neural entry's 344 attempts, a CORA-CAST program reproducing every demonstration exists for 2, both correct and among CORA's own 3 answers, and for none of the 301 wrong ones; and on the 41 query outputs, in 36 tasks, that only the neural entry answered correctly, ten times the search budget found no program, with diagnostics suggesting missing actions as a language-coverage bottleneck, not proving that none exists (Table 9; rules refined on these tasks). Table 1's *unreliable acceptance* row has microscopy counterparts in Appendix E. Beyond these, the CORA-only entry solves few hidden-test puzzles (0.83%); the positive ARC-style results are on generated or previously scored related tasks, and we inspected the public evaluation set; the biology cohort was reused, its held-input vote was designed after a first result (Appendix F), and model-based results reconstruct known functions; no CORA hypothesis has been tested in a laboratory.

9 Toward learning which capability to construct

Open-endedness is the aim: a system that keeps finding what its language lacks, builds it and checks it on problems it was not built for. The experiments do not show that; they measure where the present system breaks, and Table 2 orders the next advances from limits already measured to harder ones. The next experiment, to be pre-registered, matches conditional construction against Popper [20, 21], an inductive logic programming learner that learns from failures and invents predicates, on the 595 sparse targets at equal observations and resources; it needs a validated whole-program four-input constraint inside Popper's search, not an output filter.

10 Conclusion

Reasoning systems can fail because a needed capability is unavailable, not merely because search was too short. CORA uses task-specific evidence to constrain, verify and retain candidate executable operations. On generated tasks, demonstrations constrained the missing operation after bounded search failed (64 against 16 with another task's holes, of 95); our failures on natural tasks led us to build object-level construction, correct

on 343 of 624 held, exposed outputs against 118 for static search (57 against 30 of 315 in two dimensions). Reuse helped on 1D-ARC only, and a fixed router improved on the pre-declared order of revisions, with no advantage established over the best post hoc order. This is a bounded step toward what the organisers describe as building symbolic models of a problem on the fly and testing explicit hypotheses [14, 15, 33, 36]: the space is supplied, and the scored entry, which checks but does not construct, scored 0.83%. Construction beyond a supplied space, revision selection on natural tasks and biological validation remain open. When reasoning fails, the failure is evidence; using that evidence well requires identifying what is missing and testing whether a proposed change actually addresses it.

Code and data. The scored notebooks (CORA-only, version 364571398; neural comparison, version 364496615), a verification notebook and the GitHub release (`github.com/ChimdiWalter/Symbolic_Reasoning_LLM_ARC-AGI`; branch arc2026-sprint at commit 8ec11940, tag cora-arc2026-public-2026-11-08c) are public; the submitted version is tagged arc2026-paper-20261108.

Acknowledgements. Toni Kazic led high-level discussions on the big problems in science: what AI can and cannot do, grammar induction and program synthesis across domains, and what the next big leap could be. We thank Chris Fluharty, who started this project with us last year in a hackathon (the LLM route), and Pierce Taylor for high-level discussions on single-cell omics in biology and its bottlenecks (dispersion versus displacement), and for the joint single-cell work in the full paper.

References

[1] Tatsuya Akutsu, Satoru Miyano, and Satoru Kuhara. Identification of genetic networks from a small number of gene expression patterns under the Boolean network model. In *Pacific Symposium on Biocomputing*, volume 4, pages 17–28, 1999. doi: 10.1142/9789814447300_0003.

[2] Ekin Akyürek, Mehul Damani, Adam Zweiger, Linlu Qiu, Han Guo, Jyothish Pari, Yoon Kim, and Jacob Andreas. The surprising effectiveness of test-time training for few-shot learning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 942–963, 2025. URL `https://proceedings.mlr.press/v267/akyurek25a.html`. arXiv:2411.07279; first version titled "The Surprising Effectiveness of Test-Time Training for Abstract Reasoning".

[3] Simon Alford, Anshula Gandhi, Akshay Rangamani, Andrzej Banburski, Tony Wang, Sylee Dandekar, John Chin, Tomaso Poggio, and Peter Chin. Neural-guided, bidirectional program search for abstraction and reasoning. In Rosa Maria Benito, Chantal Cherifi, Hocine Cherifi, Esteban Moro, Luis M. Rocha, and Marta Sales-Pardo, editors, *Complex Networks & Their Applications X, Volume 1: Proceedings of the Tenth International Conference on Complex Networks and Their Applications COMPLEX NETWORKS 2021*, Studies in Computational Intelligence, pages 657–668. Springer, 2022. doi: 10.1007/978-3-030-93409-5_54. arXiv:2110.11536.

[4] Dana Angluin. Queries and concept learning. *Machine Learning*, 2:319–342, 1988. doi: 10.1023/A:1022821128753. URL `https://homepages.math.uic.edu/~lreyzin/papers/angluin88.pdf`.

[5] ARC Prize. Post on X, 16 October 2025. `https://x.com/arcprize/status/1978872651180577060`, 2025.

[6] Matej Balog, Alexander L. Gaunt, Marc Brockschmidt, Sebastian Nowozin, and Daniel Tarlow. DeepCoder: Learning to write programs. In *International Conference on Learning Representations (ICLR)*, 2017. URL `https://arxiv.org/abs/1611.01989`. arXiv:1611.01989.

[7] Rina Foygel Barber, Emmanuel J. Candès, Aaditya Ramdas, and Ryan J. Tibshirani. Predictive inference with the jackknife+. *The Annals of Statistics*, 49(1):486–507, 2021. doi: 10.1214/20-AOS1965.

[8] Robert Cregar Berwick. Computational analogues of constraints on grammars: A model of syntactic acquisition. In *18th Annual Meeting of the Association for Computational Linguistics*, pages 49–53, Philadelphia, Pennsylvania, USA, 1980. Association for Computational Linguistics. doi: 10.3115/981436.981453. URL `https://aclanthology.org/P80-1014/`.

[9] Matthew Bowers, Theo X. Olausson, Lionel Wong, Gabriel Grand, Joshua B. Tenenbaum, Kevin Ellis, and Armando Solar-Lezama. Top-down synthesis for library learning. *Proceedings of the ACM on Programming Languages*, 7(POPL):1182–1213, 2023. doi: 10.1145/3571234. Article 41; arXiv:2211.16605.

[10] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024. URL `https://proceedings.iclr.cc/paper_files/paper/2024/hash/2460396f2d0d421885997dd1612ac56b-Abstract-Conference.html`. arXiv:2304.05128.

[11] François Chollet. On the measure of intelligence, 2019. URL https://arxiv.org/abs/1911.01547.
[12] François Chollet. Post on X, 21 August 2025. https://x.com/fchollet/status/1958410017683681698, 2025.
[13] François Chollet. Post on X, 6 August 2026. https://x.com/fchollet/status/2085324762637574183, 2026.
[14] François Chollet. Post on X, 14 August 2026. https://x.com/fchollet/status/2088243704603824311, 2026.
[15] François Chollet. Post on X, 21 August 2026. https://x.com/fchollet/status/2090838046937645398, 2026.
[16] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2024: Technical report, 2024. URL https://arxiv.org/abs/2412.04604.
[17] François Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems, 2025. URL https://arxiv.org/abs/2505.11831.
[18] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2025: Technical report, 2026. URL https://arxiv.org/abs/2601.10904.
[19] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement. In *Computer Aided Verification (CAV 2000)*, volume 1855 of *Lecture Notes in Computer Science*, pages 154–169. Springer, 2000. doi: 10.1007/10722167_15.
[20] Andrew Cropper and Rolf Morel. Learning programs by learning from failures. *Machine Learning*, 110(4): 801–856, 2021. doi: 10.1007/s10994-020-05934-z. arXiv:2005.02259.
[21] Andrew Cropper and Rolf Morel. Predicate invention by learning from failures, 2021. URL https://arxiv.org/abs/2104.14426.
[22] James Cussens and Stephen Pulman. Incorporating linguistics constraints into inductive logic programming. In *Fourth Conference on Computational Natural Language Learning and the Second Learning Language in Logic Workshop*, pages 184–193, 2000. URL https://aclanthology.org/W00-0740/.
[23] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, pages 835–850, 2021. doi: 10.1145/3453483.3454080.
[24] Yu Feng, Ruben Martins, Osbert Bastani, and Isil Dillig. Program synthesis using conflict-driven learning. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 420–435. ACM, 2018. doi: 10.1145/3192366.3192382.
[25] John K. Feser, Swarat Chaudhuri, and Isil Dillig. Synthesizing data structure transformations from input-output examples. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 229–239. ACM, 2015. doi: 10.1145/2737924.2737977.
[26] Daniel Franzen, Jan Disselhoff, and David Hartmann. Product of experts with LLMs: Boosting performance on ARC is a matter of perspective. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 17657–17671, 2025. URL https://proceedings.mlr.press/v267/franzen25a.html. arXiv:2505.07859.
[27] Gabriel Grand, Lionel Wong, Maddy Bowers, Theo X. Olausson, Muxin Liu, Joshua B. Tenenbaum, and Jacob Andreas. LILO: Learning interpretable libraries by compressing and documenting code. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.19791.
[28] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 317–330, 2011. doi: 10.1145/1926385.1926423.
[29] Esther Heid, Daniel Probst, William H. Green, and Georg K. H. Madsen. EnzymeMap: curation, validation and data-driven prediction of enzymatic reactions. *Chemical Science*, 14(48):14229–14242, 2023. doi: 10.1039/d3sc02048g.
[30] Isaac Joffe and Chris Eliasmith. Vector symbolic algebras for the Abstraction and Reasoning Corpus. *npj Unconventional Computing*, 3(1):54, 2026. doi: 10.1038/s44335-026-00100-5. arXiv:2511.08747.
[31] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks, 2025. URL https://arxiv.org/abs/2510.04871.
[32] Greg Kamradt. Announcing ARC-AGI-3: A new challenge for frontier agentic intelligence. ARC Prize blog, 2026. URL https://arcprize.org/blog/arc-agi-3-launch. Published 25 March 2026.
[33] Greg Kamradt. Post on X, 3 September 2026. https://x.com/GregKamradt/status/2095600045873828013, 2026.

[34] Subin Kim, Prin Phunyaphibarn, Donghyun Ahn, and Sundong Kim. Playgrounds for abstraction and reasoning. In *NeurIPS 2022 Workshop on Neuro Causal and Symbolic AI (nCSI)*, 2022. URL https://openreview.net/forum?id=F4RNpByoqP. Mini-ARC data: https://github.com/KSB21ST/MINI-ARC.

[35] Mike Knoop. ARC Prize 2025 results & analysis. ARC Prize blog, 2025. URL https://arcprize.org/blog/arc-prize-2025-results-analysis. Published 5 December 2025.

[36] Mike Knoop. Post on X, 16 July 2026. https://x.com/mikeknoop/status/2077819572452036896, 2026.

[37] Mike Knoop. Post on X, 3 September 2026. https://x.com/mikeknoop/status/2095600676919455857, 2026.

[38] Jan Krumsiek, Carsten Marr, Timm Schroeder, and Fabian J. Theis. Hierarchical differentiation of myeloid progenitors is encoded in the transcription factor network. *PLoS ONE*, 6(8):e22649, 2011. doi: 10.1371/journal.pone.0022649.

[39] John E. Laird. Introduction to Soar, 2022. URL https://arxiv.org/abs/2205.03854.

[40] John E. Laird, Paul S. Rosenbloom, and Allen Newell. Chunking in Soar: The anatomy of a general learning mechanism. *Machine Learning*, 1(1):11–46, 1986. doi: 10.1007/BF00116249.

[41] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. GenProg: A generic method for automatic software repair. *IEEE Transactions on Software Engineering*, 38(1):54–72, 2012. doi: 10.1109/TSE.2011.104.

[42] Wen-Ding Li, Keya Hu, Carter Larsen, Yuqing Wu, Simon Alford, Caleb Woo, Spencer M. Dunn, Hao Tang, Michelangelo Naim, Dat Nguyen, Wei-Long Zheng, Zenna Tavares, Yewen Pu, and Kevin Ellis. Combining induction and transduction for abstract reasoning. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/35ee921eb17e9b4c3e73b6e2ae0d55ba-Paper-Conference.pdf. arXiv:2411.02272.

[43] Isaac Liao and Albert Gu. ARC-AGI without pretraining, 2025. URL https://arxiv.org/abs/2512.06104.

[44] Amirreza Mahbod, Christine Polak, Katharina Feldmann, Rumsha Khan, Katharina Gelles, Georg Dorffner, Ramona Woitek, Sepideh Hatamikia, and Isabella Ellinger. NuInsSeg: A fully annotated dataset for nuclei instance segmentation in H&E-stained histological images. *Scientific Data*, 11:295, 2024. doi: 10.1038/s41597-024-03117-2.

[45] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. Angelix: Scalable multiline program patch synthesis via symbolic analysis. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*, pages 691–701, 2016. doi: 10.1145/2884781.2884807.

[46] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997. ISBN 0-07-042807-7. URL http://www.cs.cmu.edu/~tom/mlbook.html.

[47] Arseny Moskvichev, Victor Vikram Odouard, and Melanie Mitchell. The ConceptARC benchmark: Evaluating understanding and generalization in the ARC domain. *Transactions on Machine Learning Research*, 2023. URL https://arxiv.org/abs/2305.07141.

[48] Stephen Muggleton and Wray Buntine. Machine invention of first-order predicates by inverting resolution. In John Laird, editor, *Proceedings of the Fifth International Conference on Machine Learning (ICML)*, pages 339–352, San Mateo, CA, 1988. Morgan Kaufmann. doi: 10.1016/B978-0-934613-64-4.50040-2.

[49] Chimdi Walter Ndubuisi. A phenotype atlas of maize lesion organization: geometry and a locked within-rater comparison. Unpublished working manuscript, 21 September 2026; aggregate results only, 2026.

[50] Chimdi Walter Ndubuisi. An image-derived segmentation pilot on maize leaf lesions. Unpublished exploratory report, November 2026; aggregate results only, 2026.

[51] Chimdi Walter Ndubuisi and Toni Kazic. Learning What to Change: Toward Failure-Grounded Co-Induction of Representations and Executable Reasoning Languages. Full technical report, University of Missouri, 2026. URL https://github.com/ChimdiWalter/Symbolic_Reasoning_LLM_ARC-AGI/blob/arc2026-sprint/paper/v2_neural_cora/CORA_FULL_PAPER.pdf. Not peer reviewed.

[52] Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. SemFix: Program repair via semantic analysis. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 772–781, 2013. doi: 10.1109/ICSE.2013.6606623.

[53] Zhuofu Ni, Andrew E. Stine, Keith E. J. Tyo, and Linda J. Broadbelt. Curating a comprehensive set of enzymatic reaction rules for efficient novel biosynthetic pathway design. *Metabolic Engineering*, 65:79–87, 2021. doi: 10.1016/j.ymben.2021.02.006.

[54] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2306.09896. arXiv:2306.09896.

[55] Peter-Michael Osera and Steve Zdancewic. Type-and-example-directed program synthesis. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 619–630, 2015. doi: 10.1145/2737924.2738007.

[56] Oleksandr Polozov and Sumit Gulwani. FlashMeta: A framework for inductive program synthesis. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 107–126, 2015. doi: 10.1145/2814270.2814310.

[57] Julien Pourcel, Cédric Colas, and Pierre-Yves Oudeyer. Self-improving language models for evolutionary program synthesis: A case study on ARC-AGI. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 49659–49688, 2025. URL https://proceedings.mlr.press/v267/pourcel25a.html. arXiv:2507.14172.

[58] Stuart Russell and Eric Wefald. Principles of metareasoning. *Artificial Intelligence*, 49(1–3):361–395, 1991. doi: 10.1016/0004-3702(91)90015-C.

[59] Julio Saez-Rodriguez, Leonidas G. Alexopoulos, Jonathan Epperlein, Regina Samaga, Douglas A. Lauffenburger, Steffen Klamt, and Peter K. Sorger. Discrete logic modelling as a means to link protein signalling networks with functional analysis of mammalian signal transduction. *Molecular Systems Biology*, 5(1):331, 2009. doi: 10.1038/msb.2009.87.

[60] Mark Schuiveling, Hong Liu, Daniel Eek, Gerben E. Breimer, Karijn P. M. Suijkerbuijk, Willeke A. M. Blokx, and Mitko Veta. A novel dataset for nuclei and tissue segmentation in melanoma with baseline nuclei segmentation and tissue segmentation benchmarks. *GigaScience*, 14:giaf011, 2025. doi: 10.1093/gigascience/giaf011.

[61] Kensen Shi, Hanjun Dai, Kevin Ellis, and Charles Sutton. CrossBeam: Learning to search in bottom-up program synthesis. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://arxiv.org/abs/2203.10452.

[62] Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. Is the cure worse than the disease? Overfitting in automated program repair. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 532–543, 2015. doi: 10.1145/2786805.2786825.

[63] Armando Solar-Lezama. Program sketching. *International Journal on Software Tools for Technology Transfer*, 15 (5–6):475–495, 2013. doi: 10.1007/s10009-012-0249-7.

[64] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit A. Seshia, and Vijay A. Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 404–415, 2006. doi: 10.1145/1168857.1168907.

[65] Ivan Sorokin and Jean-François Puget. NVARC solution to ARC-AGI-2 2025, 2025. URL https://github.com/1ytic/NVARC.

[66] Irene Stahl. The appropriateness of predicate invention as bias shift operation in ILP. *Machine Learning*, 20(1–2): 95–117, 1995. doi: 10.1007/BF00993476.

[67] Paul E. Utgoff. Shift of bias for inductive concept learning. In Ryszard S. Michalski, Jaime G. Carbonell, and Tom M. Mitchell, editors, *Machine Learning: An Artificial Intelligence Approach*, volume II, pages 107–148. Morgan Kaufmann, Los Altos, CA, 1986.

[68] Johan S. Wind. DSL solution to the ARC challenge. https://github.com/top-quarks/ARC-solution, 2020. First place, Kaggle Abstraction and Reasoning Challenge 2020.

[69] Yudong Xu, Wenhao Li, Pashootan Vaezipoor, Scott Sanner, and Elias B. Khalil. LLMs and the Abstraction and Reasoning Corpus: Successes, failures, and the importance of object-based representations. *Transactions on Machine Learning Research*, 2024. URL https://arxiv.org/abs/2305.18354. Introduces 1D-ARC; data: https://github.com/khalil-research/1D-ARC.

A Reproducibility map

result	code folder	freeze or commit	where it ran	public
CORA-only entry, 0.83%	`kaggle_entries/O_plus_v2_cora-cast-r29-v2-arc-agi-2_v1`	notebook 364571398, released byte-identical	Kaggle, scored	yes
neural comparison entry, 24.03%	`kaggle_entries/H_sf-h-c4-cast-arc-agi-2_v1`	notebook 364496615	Kaggle, scored	yes
grammar-growth replay	`experiments/capability_growth_20261027`	notebook 365118951	Kaggle	yes
construction, 64 against 38	`experiments/construction_at_scale_20261105`	freeze b17f28d025a8	three private Kaggle CPU kernels	code and records
reachability split, 13 and 17 of 31	`records/CONSTRUCTION_REACHABILITY_ANALYSIS_20261108.md`	commit 897e37a4	local CPU (post hoc)	record
learned order, 320 against 234	`experiments/guided_construction_20261106`	freeze d876eeded96c	three private Kaggle CPU kernels	code and records
held halves, 227 against 118	`experiments/natural_growth_b_20261105`	freeze 9d9d2248c511	local CPU	code and records
related benchmarks	`experiments/universality_similar_problems_20261102`	in the folder	local CPU	code and records
logic functions, 45 against 12	`experiments/transfer_logic_20261108`	GO bca541e7; results 7f7377f5	local CPU	code and records
spreadsheet text, 22 against 17	`experiments/transfer_spreadsheet_20261108`	GO fccbb0ff; outputs 8246aefe	local CPU	code and records
signalling, 863 against 805	`experiments/transfer_signalling_20261108`	GO 6ae7f578; results 7c343d56	local CPU	code and records
enzyme chemistry, 76 against 33	`experiments/transfer_chemistry_20261108`	GO 190756a6; Amendment 2 GO 849c4b50; held records c596902d	local CPU (the first held run stopped after 3 of 40 families; the amended rerun reproduced them)	code and records
ARC-AGI-3 world models, 1,204 against 902	`experiments/transfer_worldmodel_20261108`	GO ed754a3d; result cb8e58b5	private Kaggle kernel, version 5	records; outputs not published
selection test, 30 of 50	`experiments/failure_choice_20261108`	GO 87f30e19; analysis 4bc40dc6	three private Kaggle CPU kernels	code and records
selection test, revision 2, 55 against 29 of 84	`experiments/failure_choice_r2_20261108`	GO 4f88bb15; freeze fae62f8842be	private Kaggle CPU kernels (one to generate, four to measure)	records and audit
library rivals, 227 against 28 of 624	`experiments/library_rivals_20261108`	GO 3faf93d6; freeze 0ce263495c9b	local CPU	records and audit
calibrated acceptance, 430 of 605	`experiments/calibrated_acceptance_20261108`	protocol 9f283c2c; outputs 646a622a	local CPU	code and records
learned guide, 1,083 of 1,128	`experiments/spec_guide_20261108`	freeze f397e0bbbf48; GO 867d6096	local CPU	code and records; weights not public
maize trial, 16 leaves	`experiments/universality_bio_grammar_20261103`	Amendment 8 freeze 62208c3079862fa1	local CPU	freeze only; images private
gene rules, 101 against 66	companion study	9da4707166ae	local CPU	on request
chosen observations and vote	companion study	1f10e860a7b2; 0c2b7d4db1a3	local CPU	on request
conditional construction, 433	companion study	2487788cc1c8	local CPU	on request
bounded loop, 219 of 256	companion study	f284caaec787	local CPU	on request

Table 3: Where each headline result ran.

Provenance, release and licensing. Freezes are SHA-256 prefixes of the original freeze files; where the public copy was sanitised (a private folder name replaced, as for the construction and held-half freezes), `releases/2026-11-08/SANITISED_COPIES.json` maps the original's SHA-256 to the public copy's, and the release manifests list every public file with its SHA-256. A GO is the go-ahead commit that released a held run; GO, result and analysis entries are commits of the project's private research history, cited in the public audits but not on the public branch. Result records sit beside each freeze; each audit is in its study's `reviews`

folder, public for the learned-order, held-half, five new-domain, selection (both revisions), library-rivals, calibrated acceptance and learned-guide studies, while the construction study's post-score checks are in the companion study and not public (the notebook recomputes its counts). The maize trial can be reproduced only with our unpublished images. Our audits cover the ARC-style studies, the new-domain, selection (both revisions), library-rivals, learned-guide and calibrated acceptance tests and, from the companion study (a parallel line of our experiments), the gene-rule studies, the bounded loop and the noisy-data follow-ups. The GitHub release is the arc2026-sprint branch at commit 8ec11940, tagged cora-arc2026-public-2026-11-08c, with code and records of four ARC-style and five new-domain studies, the selection test (both revisions) with its full-bank control contract and checker, the library-rivals, learned-guide and calibrated acceptance tests, two Boolean review records and count-only companion aggregates, each study with its audit; the submitted version of this paper is tagged arc2026-paper-20261108, on the final release that also publishes both versions of this paper and the full paper. Code is MIT-licensed and documents are CC BY 4.0; the entries keep their 2 November grants, MIT No Attribution and CC0; none settles the checkpoint's training-data rights. A small two-input truth-table executor, identical byte for byte wherever it is used, runs on ARC grid panels and in later gene-state experiments but was not used in the gene-rule results. The full map is in the full research paper [51].

What the verification notebook checks. The public verification notebook (version 3) fetches the release at commit de74b4f9, checks every file against git's object hashes and the release manifest's SHA-256 and sizes, rebuilds the studies' prepared task files from the benchmarks at the commits the studies used, reruns the submitted CORA-only notebook on public evaluation tasks and finds its outputs equal to the saved predictions, and recomputes the construction, learned-order and public-evaluation counts from released records: 22 of its 26 checks match and none differ. The 22 are seven construction-test counts (the 95 tasks; 64, 38, 16 and 48 recovered; the 27.4-point difference; its exact paired test), the 624 held query outputs, ten learned-order counts (320, 234, 329, 318 and 323; 87 tasks gained and 1 lost; both budget curves; the exact sign test; the split by benchmark), the 172 public evaluation query outputs, the entry's 3 answers, all 3 correct, and a live rerun whose outputs equal the saved predictions on 8 of 8 query outputs of six tasks, 3 of them correct. The other 4 are marked not recomputable: the held-back-half counts 227, 118 and 343, whose sealed aggregate is not in the release the notebook fetches, and the official 0.83%, which only Kaggle computes on the hidden tasks. A count-only aggregate released at commit 5755f42f states those three held-back-half counts (`records/companion_aggregates_20261108`). Later commits up to 8ec11940 add files and change none of the 291 files the notebook checks.

Custody and audits. The five new-domain tests, both revisions of the selection test, the library-rivals test, the learned-guide test and the calibrated acceptance test were each frozen and reviewed before its held run (the library-rivals held tasks had been scored in earlier studies), and each was scored in a single held run, except chemistry: its first held run stopped after 3 of 40 families on an unhandled error, and a dated amendment, made before any held outcome was opened, led to a rerun on the same held set that reproduced the first run's 3 completed families exactly. Each audit is a separate review within the project, not external replication: written after the result record by a process separate from the study's design, it checked the saved records and found custody intact, recomputed the held counts where run records exist, and documented the wording corrections it required, or for library rivals recommended, to the study's own result record (for signalling and ARC-AGI-3 the audit failed that record's wording), none changing a held count or a pre-registered decision. This paper uses only the sentences each audit allows. Deviations on record: the selection test's three kernels were each pushed once by hand after the frozen push script stopped; in revision 2 the Kaggle login expired while the generation kernel ran, which kept running and was never pushed again, so generation ran once; and the calibrated test's audit displayed its go-ahead file's text after every held decision had been committed, against the protocol's hash-only rule.

B Proofs of the four conditional results

Write $[\![p]\!](x)$ for the output of program p on input x. A program fits D when it reproduces every demonstration, and $A(D, x^*) = \{[\![p]\!](x^*) : p \in V_{\mathcal{L}}(D)\}$.

Proposition 1. Choose $p \in V_{\mathcal{L}}(D)$, which exists by assumption. For an allowed output z, let q_z return z when its input equals x^* and run p otherwise; $\mathcal{L}$ contains q_z by assumption. Every demonstration input differs from x^*, so q_z agrees with p on every demonstration; hence $q_z \in V_{\mathcal{L}}(D)$ and $[\![q_z]\!](x^*) = z$. □ This is the program-language form of the point that a learner without inductive bias cannot classify an unseen instance [46]; it does not claim that CORA's bounded language can build these programs.

Proposition 2. Let $K_t(b)$ be the predictions retained for task t within budget b, each made at x_t^* by a fitting program of $\mathcal{L}$, and y_t the task's answer. An attempt drawn from $K_t(b)$ is correct only if $y_t \in K_t(b)$, so under any selection rule the share of tasks answered correctly is at most the share with $y_t \in K_t(b)$. Since $K_t(b) \subseteq A(D_t, x_t^*)$, that share is at most the share of tasks whose answer some fitting program of $\mathcal{L}$ predicts. □ The chain solved ≤ retained ≤ expressible is an accounting identity; its use is to say which quantity to measure first.

Proposition 3. Fitting depends only on the program, so $V_{\mathcal{L}}(D) \subseteq V_{\mathcal{L}'}(D)$ and $A_{\mathcal{L}}(D, x^*) \subseteq A_{\mathcal{L}'}(D, x^*)$, where a program that returns nothing contributes its own "no output" value. Suppose $\mathcal{L}$ fits the task, so $A_{\mathcal{L}}(D, x^*)$ is not empty. If the rule answers z under $\mathcal{L}'$, then $A_{\mathcal{L}'}(D, x^*) = \{z\}$, so its non-empty subset $A_{\mathcal{L}}(D, x^*)$ also equals $\{z\}$ and the rule answers z under $\mathcal{L}$: growth adds and changes no answer on such a task. If the rule answers z under $\mathcal{L}$, it abstains under $\mathcal{L}'$ exactly when $A_{\mathcal{L}'}(D, x^*)$ holds a value other than z, that is, when $\mathcal{L}'$ adds a fitting program that does not return z at x^*. □

Proposition 4. Take two inputs u and u' that differ only in v, with $f(u) \neq f(u')$ for the rule f. Every candidate c has $c(u) = c(u')$, so c differs from f at u or at u', and no candidate equals f, whatever the search order or budget. For the count, fix all query-selection randomness and exclude target-dependent side information. Inputs are fixed in advance or chosen only from earlier binary responses. The response histories then form a decision tree of depth at most m with at most 2^m leaves; a rule is singled out (left as the only rule consistent with what was observed) only if it is alone at its leaf, so at most 2^m rules can be singled out [1, 4]. □

Table 4 pairs each statement with the test that bears on it. Its first row shows the strict rule's trade-off: on the 1,244 query outputs it removed 21 of the 32 wrong answers and 31 correct ones, trading coverage for accuracy with no correctness guarantee.

result	test	measured	reading
Many explanations (Proposition 1)	answer whenever a program fits every demonstration, or also require the strict rule's rebuilds without each demonstration, on 1,244 query outputs of three public ARC-style benchmarks scored once before	32 of the 134 answers from fitting programs were wrong; in held logic, a four-fold rebuild check passed 54 of the loop's circuit programs, but only 33 of them were exact; the strict rule removed 21 of those 32 and 31 correct answers. In the first run on these benchmarks, given another task's demonstrations, the CORA-only entry still answered 307 of the 1,244, and 6 were correct	supported
Build before you rank (Proposition 2)	the best possible choice among the neural comparison entry's candidates, 172 public evaluation query outputs (inspected during development)	a correct answer was among the candidates for 46 and chosen for 43, so any ranking could add at most 3; a frozen ranking by CORA's contracts added 0 and lost 0	supported
Bigger language, more ambiguity (Proposition 3)	not yet run: grow a language under all-agree acceptance and count, on tasks the smaller language fits, answers turned into abstentions	no test; in one fixed imaging language, 152 programs fitted an operator's demonstrations and all-agree acceptance admitted no edit (an illustration)	untested
A missing input blocks exact recovery (Proposition 4, first part)	rules reading at most two inputs, on 595 published gene rules with a budget of 40 simulated observations per rule (reused cohort; simulations)	in logic, held and run on 8 November after the statement, splitting on the implicated input recovered 45 of 217 held benchmark functions against 12 with that input shuffled (34 to 1; file-clustered interval 0.075 to 0.241): a shown pair that differs only in one input, with different outputs, proves the function depends on that input, and a shuffled input carries no such proof (a prediction, not yet tested: the gain concentrates on splits made from single-input pairs); in genes, a consistency check, 0 of the 274 rules with three or more essential inputs; conditional construction, which reads up to four, recovered 112 of them (81 of 90 three-input); a classical learner given the same observations also reached 433	supported
Counting limit (Proposition 4, second part)	designs of observations for invented random rules over 10 variables (a simulation, not biology)	with 10 observations no design singled out more than 3% of three-input rules (the bound allows 3.9%); with 15, observations chosen where the surviving rules disagree singled out 97%, random ones 8%; in held logic, at the same probe budget, choosing three quarters of the probes where the surviving candidate programs disagreed most raised exact recoveries from 45 to 63 of 217	supported
A typed hole gives the missing piece an exact target (Section 4)	95 generated tasks, each missing one operation of a declared space of 211; three seeded repeats; pre-registered	the task's own holes 64; demonstrations alone 48 (16 gains, 0 losses); another task's holes 16, all in the one world whose removed operations compose into one another (0 of 59 in the other two); static search 38 at 3.2 times construction's charged CPU; blind enumeration 3 in every repeat, 32 in at least two, 74 in some	supported on generated tasks: discovery in a supplied space
Construction finds answers that static search at the same CPU does not (Section 4)	464 held-back tasks of ConceptARC, Mini-ARC and 1D-ARC, 624 query outputs, held out from development but scored once before	reused operations, then construction where their search failed, 343; static search with the same CPU allowance 118; construction from another task's demonstrations 10; on the two-dimensional benchmarks 57 against 30 of 315	supported on exposed related tasks
A learned order changes what a capped search finds, not what exists (Section 4)	learned against hand-set search order on the same 624 outputs, at four caps	320 against 234 at 500 counted operations per hole; 327 against 329 at 20,000, where the cap still bound on all 11 tasks whose attempts differed; a frequency table 318, a linear model 323	supported at a binding cap

Table 4: Each theoretical statement and the test that bears on it. Rows 1 to 5 are elementary conditional results, proved above under their stated assumptions; rows 6 to 8 are mechanism statements tested in pre-registered studies. Proposition 2 was stated on 27 October, before these tests; the others were added on 6 November, after some of them, so the table shows what the tests account for, not predictions made in advance. A fitting program reproduces every demonstration; all-agree acceptance answers only when every fitting program returns the same output; an essential input is one whose value can change the rule's output; blind enumeration adds the declared operations one at a time and searches the task with each. The strict-rule test of row 1 and rows 2, 7 and 8 use previously exposed tasks; rows 4 and 5 use simulations, except their held logic tests, which ran on 8 November, after all four statements.

C The pipeline in plain words

Algorithm 1 presents the common design. Search and verification ran in the scored entries; construction, admission and reuse were tested in separate research studies, including bounded integrated prototypes. The

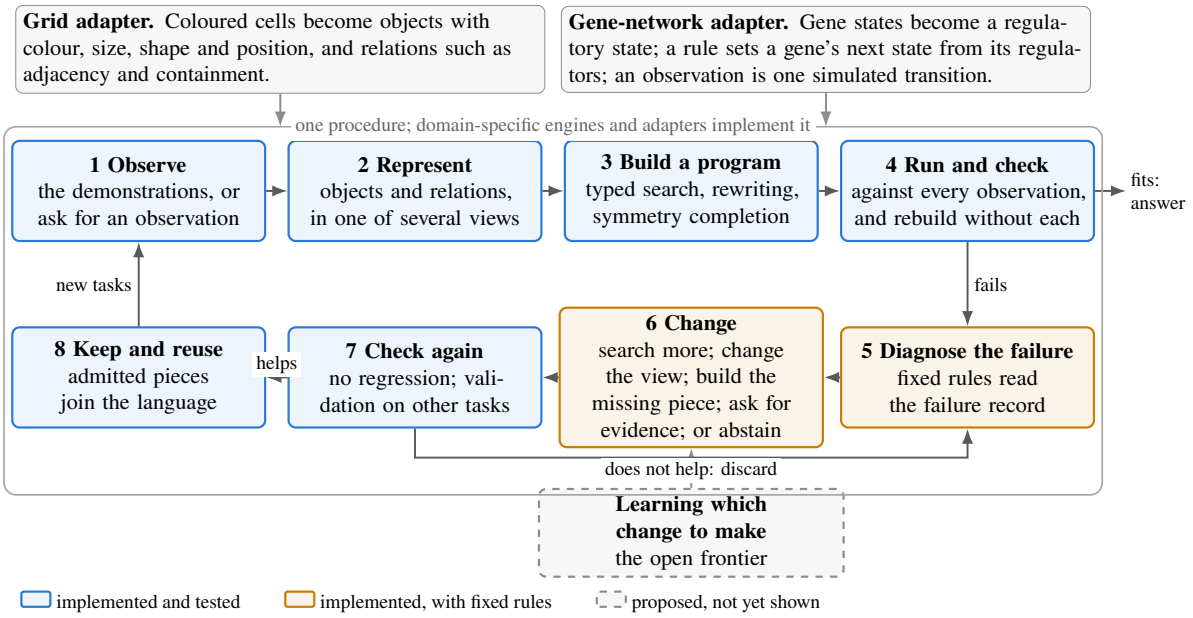


Figure 6: CORA turns a failed explanation into a missing piece. When no program fits the observations, the failure is evidence: CORA works out what the missing piece must produce, builds it, checks it on the task and on other tasks, and keeps it only if it helps. On ARC-style tasks the failure triggers construction and the typed hole comes from the demonstrations. The diagram shows what happens; how well each step works is in Figure 4 and Table 13. The scored Kaggle entry runs steps 1 to 4; the other steps ran in separate research studies. Each domain connects through its own adapter; gene-network results come from code written for biology that reuses CORA's verify-or-abstain acceptance design, not the entries' code. Colours give each step's evidence status.

Algorithm 1 CORA in plain words: answer only with a checked program, and turn a failure into a missing piece.

Require: a few observations D (the demonstrations), the queries X^*, a language $\mathcal{L}$, a budget

1: describe each observation as objects and relations ▷ the domain adapter
2: $P \leftarrow$ the programs in $\mathcal{L}$ that reproduce every observation in D, within the budget
3: **if** some $p \in P$, rebuilt without each observation in turn, still ranks that observation's output among its top two **then**
4: **return** $p(X^*)$ ▷ answer: a program that fits and is stable
5: **end if**
6: record the failure: which observation failed, which cells, a budget stop or an exhausted space
7: diagnose with fixed rules: more search, another view, a missing piece, more evidence, or abstain
8: **if** a missing piece is diagnosed **then**
9: work out from D what the piece must map (a typed hole); build candidates in a declared space
10: keep a candidate only if it is new, fits the task, breaks no solved task and passes other tasks
11: add the kept piece to $\mathcal{L}$ and go back to line 2
12: **end if**
13: **return** abstain ▷ no checked program: decline to answer

complete diagram was not deployed as either scored ARC-AGI-2 entry.

D More negatives, defects and earlier selection tests

Why learned guidance tied a frequency table (post hoc, development data). On the source halves the table already tried a working edit first in 95.6% of the 633 holes where one existed, though the failure features named the kind of edit that works in 98.1% of them, so the open problem is building missing edits, not ordering existing ones; the scorer loaded each benchmark's whole solutions file, but no held output was used

what we measured	what it taught us	what to change next
frequency (318) and linear (323) orders matched the learned guide (320)	the table already tried a working edit first in 95.6% of 633 holes (post hoc, development data)	build missing edits; test guidance where the right edit is rare, as the held learned-guide test then did at a binding budget (Section 6)
strict checks admitted nothing on noisy data; frozen follow-ups admitted no Perturb-seq rule, and the microscopy follow-up failed technically, with no result	a zero-error check can reject a valid relationship when observations contain noise; whether noise caused these failures is not established; on one donor a predeclared secondary neighbour guard turned 58 corrections and 65 harms into 36 and 12 (exploratory)	count each gene's correctable events before a run; repeat the guard on a second donor against a random filter of matched size
on 16 of our maize leaves a segmenter trained on cell images scored F1 near zero and CORA abstained on every leaf	the tested local edits did not rescue this representation	detect a collapsed baseline and change the representation first
the neural entry was correct on 41 query outputs that CORA did not answer; ten times the budget found no program (post hoc, exposed)	a tenfold increase in search did not recover fitting programs; post hoc diagnostics suggest missing actions and inadequate object views contribute, although deeper search within the existing language cannot be ruled out	build actions that draw on the background, edit object parts and assemble grids
an order counted from ARC training tasks did not help gene rules (177 against 194 for a hand-set simplicity order)	verify-or-abstain acceptance and evidence requests carried over; puzzle content did not	learn priors inside each domain
on the 120 public evaluation tasks the learned order missed, at 500 counted operations per hole, the one task the hand-set order answered (both answered it at 20,000); on the two two-dimensional benchmarks the two were close (45 against 43 of 315 query outputs; exposed)	the learned order's gain is on one-dimensional tasks	test learned guidance on two-dimensional and real tasks
the official score is 0.83%	on exposed sets coverage, not precision, looks like the main limit; the hidden-task split is not observed	grow coverage by construction under the CPU cap
reuse lost on the two-dimensional benchmarks (11 against 30 of 315)	one-dimensional operations rarely fit two-dimensional tasks	build per task; reuse only after a check on another family
a whole-program library lost to static search given its whole cost (37 against 56 of 624)	storing found programs bought less than the same compute spent on search (it beat the starting language, 37 against 6)	admit an entry only if it beats static search given its discovery cost on tasks it was not built from
a fixed diagnosis loop tied a build-then-ask schedule (219 each of 256 authored functions), and a failure-reading router tied a fixed order (30 each of 50 generated tasks)	with budget for every change a fixed order catches up; construction rejects an inapplicable hole after about 900 calls	a change that cannot cheaply check whether it applies
spreadsheet text: 22 against 17 and 17 of 53, neither pre-registered comparison significant; plain search at 16 times the budget also 22	with a single held-out example for most tasks, a gain of about five tasks in 53 was not detectable, and rebuilding a piece from one remaining example was too weak an acceptance test	more held-out examples per task and an acceptance rule suited to two or three training examples
the ARC-AGI-3 world-model probe did not pass its pre-registered test (interval over games 0.0 to 0.232; gains in 3 of 12 games)	gains from growing the language appeared only where a game showed a regularity that the piece kinds could express	more games or a wider set of piece kinds

Table 5: More of what did not work, continuing Table 1: the learned guide, noisy data, the maize leaves, the neural entry's outputs, the ARC-counted order, the learned order on two-dimensional and real tasks, the official score, reuse in two dimensions, the whole-program library, the fixed diagnosis loop, spreadsheet text and the ARC-AGI-3 probe.

or scored.

Choosing what to observe. Choosing what to observe extends the choice of change to interactive settings like ARC-AGI-3 [32], where an initial world-model probe did not pass its test.

Why real ARC-AGI-2 cannot yet test the choice of change. All CORA programs together solve 2 of its 120 public tasks, so even if one method solved both and another neither, the exact two-sided p would be 0.5. A protocol for a real-puzzle version has passed review as a draft and has not run.

An early-stop defect. A check on training demonstrations exposed a defect: when one way of cutting a grid into objects held more than 64 objects, the whole search stopped, so a limit on one representation must refuse only that representation. A minimal correction that does so passes regression tests on invented grids;

no corrected run on real tasks exists yet.

The selection test in two revisions. Where one change can check cheaply whether it applies, the best fixed order matched the oracle and choosing added nothing (revision 1); where a change cannot, no fixed order of the bank reached the oracle (best 44 against 55 of 84), and the pre-registered router did (revision 2), beating the pre-registered order; its advantage over the best fixed order chosen after the results is not established.

The selection test, revision 1, in its audit's sentences. The test varies only the order of the same changes at equal cost; each task has one known cause, a missing operation or a starting-language program beyond the base budget (a depth task). "In a pre-registered test on 50 held controlled generated tasks (transformation tasks built from ARC training input grids, not ARC puzzles), a router that reads CORA's failure record gave the same answers as a static bank of the same two changes in a fixed order on every task: at the binding budget of 3.5 million counted primitive executions both solved 30 of 50, as did an oracle told each task's cause (exact McNemar p = 1); the pre-registered outcome was none." "Failure evidence beat a random order of the same changes (30 against 20 tasks; 10 gained, none lost; Holm-adjusted p = 0.006) but not the best fixed order." "The reason is in the bank: typed-hole construction rejects an inapplicable hole after about 900 calls (897 to 913 on all 18 held depth tasks), so trying it first costs almost nothing, and the best fixed order already matches the oracle at the binding budget and above." "The bank's advantage over spending the same budget on more search pointed the same way as on the source set (10 against 2 tasks) but did not meet the pre-registered rule (p = 0.039; family-clustered interval -2.2 to +34 points)." "At a quarter and a half of the binding budget, extra search alone solved more tasks than the bank and the oracle (9 against 7, and 19 against 15), because construction applied but could not finish within the budget; no router in this test used the budget." "A learned router trained on the source families did worse than the fixed order (27 against 30; three tasks of one held family sent to search first and lost; Holm-adjusted p = 0.5)." Here the bank is the static bank of the same two changes in a fixed order.

The selection test, revision 2, in its audit's other sentences (pre-registered; scored once; audited). Revision 2 added a search over alternative object views; Section 6 gives its primary result and its limit against the best fixed order. "The whole gain came from the 28 wrong-object-view tasks, whose fix, a search over alternative object views, cannot cheaply check whether it applies; at every budget the router solved exactly the tasks solved by an oracle router told each task's injected cause." "The router also beat its controls: the same rule applied to another task's failure record (18 tasks against 0) and a random order of the same bank (22 against 4), both holding after Holm adjustment; a router learned from the source families only solved 54 of 84 at the binding budget." "The advantage shrank as the budget grew, from 26 tasks against 0 at the binding budget to 3 against 0 at twice it, as predicted before the held tasks existed." "The size fact separates the causes because of how the tasks were generated, and it comes from the demonstrations rather than from the failed search, so the result shows that choosing the order pays when the evidence separates the causes, not that CORA's failure record does so on natural tasks." "The pre-registered static order was chosen on a source set with fewer view tasks and scored below a random order on the held mix (29 against 37), and two of the three held causes reuse the previous revision's held families with fresh input grids." Its audit is `experiments/failure_choice_r2_20261108/reviews/POST_SCORE_AUDIT.md`.

Learned Boolean selection (held study not run). The held study was not run: its pre-declared gate found no room for a selection effect on the development tasks, where every method tied; with each method charged only for its own work, a fixed schedule could try the whole bank of candidate input sets within the budget, so diagnosis had little to save. "Choosing what to change can pay for itself only when the bank of options is too large for a fixed schedule to cover within the budget; the next test needs such a bank, for example rules over many more candidate inputs, with each method charged only for its own work and the stopping gate fixed in advance." Here counted operations are a deterministic count of elementary checks, not time.

Earlier selection tests. Failure evidence did not help choose a repair in two earlier generated-task tests: on 288 groups it changed the share of correct choices by −0.6 points (interval −2.6 to +1.4); on 27 repair tasks, authentic, failure-blind and shuffled failure evidence each certified 3, and the entry built on that design

(submission 57354496) scored 0.00%. Lesson: a failure descriptor helps only if it carries information the demonstrations lack. Direction: features taken from the failed search itself.

E The evidence boundary in biology

Conditional construction in full. In a third protocol, conditional construction, which builds a rule over more inputs from two simpler rules (a cofactor split), recovered 433 rules exactly at 40 observations, against 321 for the complete two-input rule language on the same observations. All 112 additional rules have three or more inputs (81 of 90 with three inputs and 31 of 184 with four or more), which the two-input language cannot express, so this shows executable coverage, the biology counterpart of building a missing operation; a classical learner given the same observations tied it at 433, so it is not an advantage over classical reconstruction, and 91 wrong programs still fitted every observation (audited). The run does not test whether a failure, rather than always using the larger language, triggered the gain.

An operator order counted from 22 exact Boolean relations in ARC training demonstrations recovered 177 of 595 gene rules, against 168 for plain index order, 182 for the same counts reassigned at random and 194 for a hand-set simplicity order; its gain over index order did not survive correction for multiple comparisons, and it lost 20 targets and gained 3 against the simplicity order, so priors should be learned inside each domain, while verify-or-abstain acceptance and evidence requests carried over. Three levels of validity stay apart: a rule can execute and fit the examples (computational), make frozen predictions that match independent measurements (predictive), or be supported as a mechanism (mechanistic), which needs a designed intervention that separates it from its rivals and then molecular evidence for its route. CORA's biology reaches the first level and has been tried on measured data without a robust success (Table 7). For noisy continuous data we state a hypothesis, not a law: noisy domains need relational guards and an acceptance bar that weighs a prior; whether noise causes these failures is untested, and the prior-weighted bar has not been tested apart from its prior.

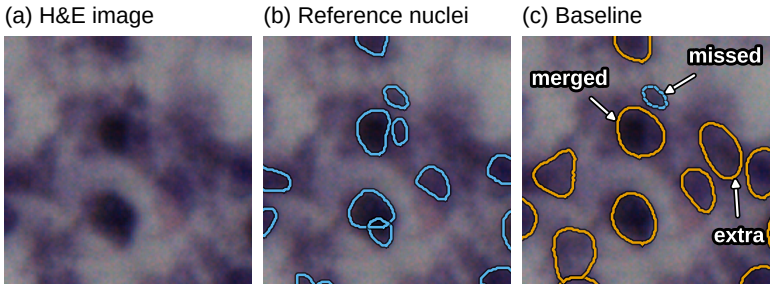


Figure 7: Melanoma nuclei from a NuInsSeg calibration image [44] (CC BY 4.0), with the baseline's merged, missed and extra errors (measured data; a domain study, not a CORA success). CORA's strict check made no held-out edit; a calibration-informed tolerant rule made 32 accepted edits on held NuInsSeg images, 10 beneficial, 16 harmful and 6 neutral, and no target passed its control gates; a run with every operator and no admission step made 4,000 edits, 1 helpful and 7 harmful.

ARC visual reasoning	biological reasoning
input grid; output grid	current gene state; next state or perturbation response
object; relation	gene or protein; regulation or dependency
transformation rule; missing operation	update rule; missing interaction or regulator
run and compare a program	simulate a rule and compare it with observed states

Table 6: The mapping between the domains. It holds only through an adapter that declares meaning; spatial adjacency does not become regulation by renaming.

What the biology shows. These results reconstruct published computer models from simulated observations. None is an independent prediction or a discovery; frozen predictions, tested against independent measurements and interventions, are still needed.

stage	current CORA evidence	what remains
construct executable biological rules	demonstrated for published Boolean models (published-model reconstruction)	more complex and noisy relationships
identify relevant regulatory variables	inferred regulators: 101 against 66 of 595 in a replication frozen before its single run, every exact recovery a one-input rule; chosen observations: 320 against 291	stronger comparisons and broader independent data
grow the executable rule language	authored functions: 219 against 38 of 256, and a fixed build-then-ask schedule also 219; published models: 433 against 321 of 595, and a classical learner also 433	growth that the system chooses and that beats fixed alternatives on natural models
predict independent biological outcomes	held inputs of simulated models only (conditional construction 2,070 against 1,400 for the two-input language, of 2,380, a classical learner also 2,070; post hoc, 2,200 against 2,153); on measured data no rule has yet shown independent predictive validity	robust prediction on independent measurements, with frozen predictions
transfer a learned revision strategy from ARC	not established: an order counted on ARC gave 177 against 194 for a hand-set simplicity order	beat strong native and fixed methods with a frozen transferred policy
confirm a new hypothesis in the laboratory	not demonstrated; no hypothesis qualifies	prospective experiments

Table 7: The evidence boundary in biology. Published-model results reconstruct computer models from simulated observations; none is a discovery, and no CORA rule has been tested in a laboratory.

F What the numbers mean

Each row keeps its own unit (tasks, query outputs, rules, functions, input states or attempts), and counts in different units are not comparable. A post hoc diagnostic is not proof that the language cannot express a solution; the neural checkpoint is contaminated for the public evaluation set, and the diagnostic rules were refined on those tasks.

number	what is counted	against what	why it matters	label
0.83%	Kaggle public-leaderboard score on 120 hidden tasks (either of two attempts counts)	0.00% for every earlier CORA entry	CORA alone solves few hidden-test puzzles; how many it answered is not reported	official score
81.5% to 95.8%	answered query outputs that were correct (1D-ARC 295 of 308, Mini-ARC 19 of 20, ConceptARC 44 of 54; answered 308 of 615, 20 of 149 and 54 of 480)	at most 2.4% with another task's demonstrations	answers are precise and depend on the task's evidence	exposed related benchmarks
64 against 38 of 95	generated tasks recovered	static search in the starting language, which used more CPU	typed holes computed from the demonstrations find missing operations	generated tasks
227 against 118 of 624	held query outputs correct	static search in the starting language	reuse works within 1D-ARC only	held, exposed
320 against 234 of 624	held query outputs correct at 500 counted operations	the hand-set order	a learned order spends a small budget better	held, exposed
1,083 of 1,128	held tasks generated from ARC-AGI-2 public training input grids, solved at 125 counted operations each	the hand-set order 953; a frequency table 881; a linear model 1,018	search efficiency at a binding budget, not new reach	held, generated; pre-registered; audited
2 of 172	public evaluation query outputs correct by construction	0 for every other condition	reaches a real puzzle the CORA-only entry already answers; one task, inconclusive	exposed public set
2 of 344	neural attempts that a CORA program reproduces from the demonstrations	none of the 301 wrong attempts	CORA can check a neural answer only where its language reaches	exposed public set, post hoc
41 in 36 tasks	query outputs the neural entry answered correctly and CORA did not answer	no program at ten times the budget	tenfold search found no fitting program; diagnostics suggest missing actions and views, not a proof	exposed public set, post hoc
101 against 66 of 595	published gene rules rebuilt exactly	three controls with no regulator preference or shuffled regulators, exact on the same 66	regulators inferred from the same observations help choose among fitting one-input rules	published-model reconstruction; first scoring of this cohort
320 against 291 of 595	exact rules after 20 chosen observations	20 random observations (a permuted-variable control: 297)	better observations identify rules	published-model reconstruction, reused cohort
2,200 against 2,153 of 2,380	held input states of the simulated models predicted correctly	random choice, same vote	a researcher-made vote recovered the loss (post hoc; a fill of zero reverses it)	published-model reconstruction, reused cohort; a first scoring attempt stopped at its 120-second limit after 235 of 595 targets, and those rows were not used
219 against 38 of 256	authored Boolean functions recovered exactly	the two-input language; a fixed build-then-ask schedule also 219	the larger language gives the gain; diagnosis adds no accuracy	synthetic demonstration (authored functions)

Table 8: What the headline numbers mean.

public evaluation, 172 query outputs	count
neural comparison entry correct, either of two attempts	43 (33 on the first attempt)
CORA-only entry answered; correct	3; 3
neural attempts that a CORA-CAST program explains	2 of 344, both correct; 0 of the 301 wrong
neural entry correct, CORA no answer	41 query outputs in 36 tasks
a CORA-CAST program at ten times the budget	none
first gap: a missing action	35 tasks (in 14, object views also fail)
first gap: a missing guard	1 task

Table 9: Where the neural comparison entry and CORA each fall short (Section 8). Post hoc, exposed public evaluation, not an official score. *Explained*: a CORA-CAST program reproduces every demonstration and outputs the attempt. Ten times the budget multiplies executions, kept candidates and CPU caps, with CORA's language fixed; this tenfold increase in search did not recover fitting programs, and post hoc diagnostics suggest missing actions and inadequate object views contribute, although deeper search within the existing language cannot be ruled out. The diagnostics read only the demonstrations and queries, and their rules were refined while being run on these tasks. Both tasks CORA answers shaped its design, and the neural checkpoint is contaminated for this set. CORA's third answer could have raised the neural entry from 43 to 44 correct, but that entry's own CORA step stopped at its 15 CPU-second limit on that task.

G The six award criteria

Accuracy: 0.83% on the hidden tasks for the CORA-only entry, which submits program-derived answers only after checking; its precision on the exposed benchmarks covers answered queries only, and how the hidden score splits into precision and coverage is not observed. **Universality:** the procedure, reimplemented through adapters, ran in thirteen domain families (Table 14 shows twelve; the full paper adds drug response, a negative); held tests in five new domains gave mixed results: a logic gain under the shuffled control, a signalling gain concentrated in one edge, spreadsheets not significant, chemistry 76 against 75 of 200 held substrates for nearest neighbour, no advantage shown and the control not independent, and an ARC-AGI-3 probe that did not pass. **Progress:** construction and reuse beat static search (64 against 38 of 95 generated tasks; 343 against 118 of 624 exposed held outputs). **Theory:** four conditional results, proved in Appendix B, three supported by tests and one untested (Table 4); Section 6 measures when choosing the change can pay. **Completeness:** materials exist for several principal experiments (frozen protocols, custody, audits, negatives and a verification notebook), and some parts are not reproducible from the public release (Appendix A). **Novelty:** the contributions of Section 1; Table 10 adds each next test.

criterion	evidence	what it means, and the next test
Accuracy	CORA-only entry 57902131: Kaggle public-leaderboard score on 120 hidden tasks, 0.83%; 3 of 172 public evaluation query outputs answered, all correct; neural comparison entry 57867631: 24.03%, our re-implementation of NVARC's method with CORA's second attempt, 3.6 points below NVARC's own notebook rerun by us (27.64%, not ours)	precise where it answers on exposed sets; how the hidden 0.83% splits into precision and coverage is not observed; coverage is the goal; the hybrid's hidden CORA increment is unknown
Universality	the procedure reimplemented through adapters in thirteen domain families (the full paper adds drug response, a negative); held tests in five new domains gave mixed results (logic gained under the shuffled control, signalling in one edge, spreadsheets not significant, chemistry 76 against 75 of 200 held substrates, no advantage shown, control not independent, the ARC-AGI-3 probe not passed); precision 81.5% to 95.8% on three related benchmarks; 101 against 66 and 320 against 291 of 595 published gene rules, from separately written biology code; logic 45 against 12 of 217 (shuffled description; reconstructions)	verify-or-abstain acceptance and evidence requests carried over; puzzle content did not (177 against 194); list functions and spreadsheet text inconclusive; nothing admitted helped in three measured-data domains; next, a fresh gene-model cohort, not feasible today (102 targets from 13 lineages)
Progress	construction 64 against 38 of 95 (48 from the raw demonstrations); 343 against 118 of 624, 57 against 30 of 315 in two dimensions; 320 against 234 of 624 at 500 counted operations per hole (search CPU 115 against 106 seconds; every held task scored once before by the submitted entries)	discovery moved inside supplied spaces; at a binding budget a learned guide solved 1,083 of 1,128 held generated tasks against 953 for the hand-set order and 881 for a frequency table, search efficiency, not new reach; where a change cannot cheaply check whether it applies, the pre-registered router reached the oracle (55 of 84) and no fixed order did (best 44), an advantage not established against that best order
Theory	four elementary conditional results, three tested and supported and one untested, and three tested mechanism statements (Table 4)	they account for the measured pattern under their assumptions; the next test is growth under all-agree acceptance
Completeness	scored notebooks, a verification notebook, frozen protocols, failed attempts, controls, costs, our own audits and a reproducibility map (Appendix A); code and records of four ARC-style studies and five new-domain studies public	private kernels and the biology studies' code are described, not linked
Novelty	controls that change only the evidence a build reads (64 against 16, and 32 by three-repeat majority, of 95; in logic 45 against 12 of 217, reconstructions); reachability (13 and 17 of 31); a measured condition for when choosing the change can pay; one adapter recipe with held tests and negatives	known ingredients, with evidence that earlier work has not, to our knowledge, shown (Appendix H); no control removes the admission checks

Table 10: The six Paper Track criteria.

The organisers' stated aims. The organisers' public posts describe strong systems as building symbolic models of a problem on the fly [14, 15, 36], testing explicit hypotheses [33], acquiring skills efficiently [12] and using learned guidance inside a symbolic outer loop [13]; they associate strong systems with learning

outside model weights while open-ended invention remains unsolved [37], and with working openly [5, 32]. CORA advances the first two in a bounded way, building an executable operation from a supplied space while solving a task and admitting program-derived answers only after an explicit check; in logic, choosing observations where surviving candidates disagreed raised exact recoveries from 45 to 63 of 217 held functions at the same probe budget. On held generated tasks a learned guide spent a binding search budget better than a hand-set order and a frequency table, without new reach, and the work is open (public code; a notebook reproducing 22 of its 26 checks). On the 41 query outputs that only the neural comparison entry answered correctly (post hoc, exposed public evaluation, not an official score), a larger search still found no fitting program and diagnostics suggested missing operations; one research direction is a learned proposer for typed holes, followed by CORA's explicit checks. It did not pass its ARC-AGI-3 probe; open-ended invention beyond a supplied space is a direction, not a result; and on controlled generated tasks a fixed router improved on the predeclared order, while its advantage over the best post hoc order remained inconclusive and natural-task selection remains untested (Section 6).

H Related work in detail

Table 11: How earlier systems use a failure, whether their language grows, how they check, and where CORA differs. CORA's construction step: once a bounded search fails, typed holes are computed from the demonstrations; an operation is selected for them from a supplied space (211 whole-grid operations in the generated-task test) and admitted only if a bounded search in the starting language does not reproduce it, it fits its task, it breaks no solved task and it fits another unsolved task; admitted operations are reused. CORA-CAST answers only when its program fits every demonstration and, rebuilt without each one, still ranks that demonstration's output among its top two.

prior work	what a failure is used for	language: fixed or grown	how it checks	CORA: how close, and the difference
Bias shift [67]; predicate invention [48, 66], also while learning from failures [21]	Bias shift changes a learner's bias, including the terms of its description language, as it gains experience. Predicate invention adds predicates when the vocabulary is insufficient; for some restricted languages it does not help when learning fails.	Grown: new terms or predicates.	Consistency with the examples.	The closest lineage for growing a language when it falls short. CORA's addition is the evidence: controls that keep the search and checks fixed and change only what the build reads, the task's own holes against another task's holes (64 against 16 of 95 generated tasks) and against the same operations tried blind (32), and in logic the failure description against a shuffled one (45 against 12 of 217; reconstructions of known functions).
Learning from failed programs: Popper [20]; conflict-driven synthesis, Neo [24]	Pruning. Popper: a program that is too specific (misses a positive example) or too general (covers a negative one) becomes constraints that prune its specialisations or generalisations. Neo: the cause of a failed partial program becomes a learned constraint that prunes others failing for the same reason.	Fixed: background predicates and syntactic bounds (Popper), a given domain-specific language (Neo).	Popper: entails every positive and no negative example. Neo: satisfies the input and output examples.	Popper and Neo read each failure; on ARC-style tasks CORA does not: a failed search only triggers construction, and the hole comes from the demonstrations. CORA adds an operation instead of pruning, and keeps it across tasks only after checks on other tasks.
Growth where analysis fails: Berwick's grammar acquirer [8]; missing grammar rules from failed parses [22]; chunking in Soar [39, 40]; DeepCoder's sort-and-add search [6]	Trigger, and in one case location. Berwick: when no known rule applies, build one new rule that does, and save it. Cussens and Pulman: infer by abduction the edges a failed parse lacked, build rules that would supply them, then generalise. Soar: an impasse starts reasoning in a subgoal, and chunking compiles that reasoning into rules. DeepCoder: whenever search fails, add the next most probable function and restart.	Grown: grammar rules (Berwick; Cussens and Pulman) or production rules (Soar). DeepCoder: a wider active subset of a fixed language.	Berwick: the new rule must apply where no known rule did. Cussens and Pulman: candidate rules are evaluated on the data and added when the user commits. Soar: not assessed. DeepCoder: the program must fit every example.	The trigger is theirs: like Berwick's acquirer, CORA builds one piece when its language fails; unlike Cussens and Pulman, it does not use the failed attempt to locate the gap. Its additions are the checks before keeping (new, fits, breaks no solved task, fits another unsolved task) and the controls above. No control isolates the trigger itself.

prior work	what a failure is used for	language: fixed or grown	how it checks	CORA: how close, and the difference
Sketching with holes: Sketch [63, 64]; type-and-example-directed synthesis, Myth [55]	Sketch: a sketch with no valid completion is reported as buggy; counterexamples refine its inputs. Myth: a guessed term that disagrees with an example is discarded.	Fixed: the programmer writes the sketch (Sketch); a typed functional language (Myth).	Sketch: equal to a reference program on every input. Myth: consistent with every example.	The same idea: a hole with a specification, which Myth also derives by pushing examples into holes. CORA's sketches are three fixed shapes, not written by a programmer; the hole is filled from a supplied space, and the filler becomes a reusable operation. With a few demonstrations and no reference program, CORA's checks show consistency, not correctness.
Deduced hole specifications: witness functions in FlashMeta [56], which re-implements FlashFill [28]; λ^2 [25]; inverse semantics on ARC [3]	Pruning: a sub-specification that no program meets yields an empty set, and that branch is dropped (FlashMeta); deduction refutes hypotheses that cannot be completed (λ^2).	Fixed: a domain-specific language whose designer supplies the witness functions (FlashMeta); a functional language with combinators (λ^2).	Every kept program fits all examples by construction; a ranking picks one (FlashMeta); λ^2 returns the simplest program that fits.	The closest precedent for the hole: undoing one invertible operation is a simple witness function. The one-operation sketches beat the demonstrations alone (64 against 48 of 95 generated tasks; 16 gains, no losses), which measures this known idea inside CORA. Where FlashMeta drops a branch no program meets, CORA selects an operation for it from a supplied space and, if admitted, keeps it for later tasks.
Library learning: DreamCoder [23]; LILO [27]; Stitch [9]	None: unsolved tasks are retried in later cycles; new routines are compressed from programs found for solved tasks.	Grown: a library of routines.	DreamCoder: a program must predict every observed output; routines are chosen by a Bayesian objective. Stitch: abstractions chosen by how much they compress a corpus.	Both grow a language, and both change what a bounded search reaches; on drawing and tower-building tasks, DreamCoder's library learning raised its share of held-out tasks solved. CORA builds from unsolved tasks' holes and admits after checks on other tasks; with no matched comparison against these systems, no advantage over them is claimed, and the same-size library comparison below (227 against 28 of 624 outputs) is not a general method comparison. Our library of whole programs beat the starting language (37 against 6 of 624 held query outputs) but lost to static search given its whole cost (37 against 56); that tests our library, not DreamCoder's.
Counterexample-guided loops: CEGIS [63, 64]; abstraction refinement, CEGAR [19]	Refinement. CEGIS: each counterexample input from the verifier joins the synthesizer's inputs. CEGAR: a spurious counterexample splits an abstract state.	CEGIS: a fixed space of hole values. CEGAR: the abstraction grows finer.	CEGIS: a verifier with a complete specification. CEGAR: model checking; each refinement eliminates the spurious counterexample.	CEGAR is the closest analogue to a failure changing the representation, and it uses the failure more: the counterexample places and justifies the change. ARC supplies only the demonstrations, with no verifier that can return new counterexample inputs. Direction: let the failed search place the hole, and test that against holes from the demonstrations alone.
Program repair: GenProg [41]; SemFix [52]; Angelix [45]; self-repair by language models [10, 54]	Localisation and repair: the failing test or execution result locates or describes a fault, then a patch or a rewrite follows.	Fixed: repairs edit one program; nothing is kept as a language operation.	Pass the tests; patches that pass the tests can still overfit them [62].	SemFix is close: a located hole, a derived specification, a filler from a fixed space. Repair patches one program; CORA's filler becomes a language operation, checked on other tasks and reused. No reported control removes CORA's admission checks.
ARC program search and test-time training: the 2020 Kaggle winner [68]; test-time training [2]; NVARC [65], built on the ARChitects' method [26]; held-out demonstrations on ARC [30]; jackknife+ [7]	None. The 2020 winner keeps the programs that solve the most demonstrations, then the shallowest. Test-time training trains on leave-one-out tasks built from the demonstrations. Joffe and Eliasmith hold out each demonstration in turn to choose which object properties a rule reads.	2020 winner: fixed and hand-written. Test-time training: no program; a network's weights adapt to each task.	Fit, then ranking (2020 winner); votes across transformations of the task or the model's own output probabilities (test-time training); jackknife+ gives intervals with a coverage guarantee for exchangeable data.	CORA-CAST is in the program-search line and abstains when a program fails the rebuild check. Holding out one demonstration at a time is shared with test-time training and with Joffe and Eliasmith, but CORA-CAST uses it to decide whether to answer; unlike jackknife+ it gives no coverage or precision guarantee. Our neural comparison entry, which re-implements NVARC's method, scored 24.03% against 0.83% for the CORA-only entry.

prior work	what a failure is used for	language: fixed or grown	how it checks	CORA: how close, and the difference
ARC Prize 2025 paper awards: TRM [31]; SOAR [57]; CompressARC [43]	TRM: no explicit failure step; a small network refines a latent state and its answer over recursive steps. SOAR: a language model refines programs using their execution on the demonstrations, and its attempts are relabelled as correct programs for the tasks they do solve, then used for fine-tuning. CompressARC: a network trained on the one puzzle to minimise its description length.	TRM and CompressARC: no program, a network. SOAR: Python, with model weights updated between rounds.	TRM: a vote over augmentations. SOAR: programs run on the demonstrations. CompressARC: the most frequently sampled answer.	SOAR is the closest ARC work that learns from failed programs, but what it learns lives in weights; CORA's change is one explicit operation that must pass checks and can be inspected and reused. TRM refines inside a fixed network and always gives an answer; CORA-CAST abstains when its check fails.

Library rivals (pre-registered; scored once; audited). Its audit's sentences, which Sections 1 and 5 shorten: "On 464 held tasks (624 test outputs) from ConceptARC, Mini-ARC and 1D-ARC, held out from the libraries' construction but scored in earlier studies, we added three kinds of library of the same size per benchmark (2, 0 and 19 entries) to the same base program search: operations kept from failures on the source halves, a single-step compression of solved programs, and random whole-grid operations (drawn twice). Each entry entered as one operation of equal cost, under the same budget of 150,000 primitive executions and two attempts." "The failure-kept library produced 227 correct outputs against 28 for the compression library. 218 tasks were right only under failure-kept and 22 only under compression (exact McNemar $p < 0.000001$; family-clustered 95% interval for the difference of 199 outputs: 89 to 294)." "Both random libraries scored 6, the same as the base search alone on every task, and none of their entries appeared in any program the search found." "By benchmark, failure-kept scored 9 of 240 outputs on ConceptARC, 2 of 75 on Mini-ARC and 216 of 309 on 1D-ARC, against 4, 2 and 22 for compression and 4, 2 and 0 for the base search alone." "Failure-kept led compression by 194 outputs on 1D-ARC (interval 83 to 289), and the ConceptARC difference of 5 outputs was inconclusive (Holm-adjusted p 1.0). Mini-ARC, where no library has entries, is a tie by construction." "Compression's only gain over the base search was 22 tasks of the 1D-ARC flip family, where the failure-kept library got nothing right, so in this setting the two libraries are complementary." "Read descriptively, this is a low-success regime. The source halves offered few solved programs to compress (on 1D-ARC, 23 of 306 tasks, only 2 distinct programs, all found with four times the base budget), while on the same 1D-ARC source half the failures still supplied specifications from which the builder admitted 19 operations, 12 of which fired on held tasks." "These results have firm limits: the failure-kept arm repeats an earlier study's object-reuse arm, whose predictions it matches on all 464 tasks; the design was written knowing that study's held results; construction work was not matched (about 9,300 CPU-seconds for the 1D-ARC failure library against seconds for the compression); the compression is a single step with no holes; the failure material cannot be separated from the object-level operation space; and this is not a general head-to-head of library-learning methods." Its audit is `experiments/library_rivals_20261108/reviews/INDEPENDENT_POST_SCORE_AUDIT.md`.

I Details kept from the main text

The narrative revision of 8 November 2026 condensed some main-text passages; their full statements are kept here.

I.1 Tables moved from the main text

The other held tests of the rebuilt loop. Table 12 gives the spreadsheet, signalling, chemistry and ARC-AGI-3 tests with their controls and limits; Section 7 summarises them.

domain, held test	result	control and limit
spreadsheet text: SyGuS (real tasks)	22 against 17 (plain search at four times the budget) and 17 (another task's failure) of 53 (46 real forum problems)	a null: Holm-adjusted $p = 0.125$ each; plain search at 16 times the budget also 22: mainly faster search
cell signalling: the DREAM4 subset of the liver network of [59], one prior edge removed (synthetic tasks)	863 against 731 (static) and 805 (shuffled description; 50 tasks won, 11 lost) of 1,000 held conditions	44 of the 58 gains over the shuffled control from one of six removable edges, mek12 to erk12; elsewhere 2.0 points, not significant; the static program cannot express the removed edge; repeats the development design on new draws; the shuffle across time points is weaker
enzyme chemistry: EnzymeMap [29], 3 examples per family (real reactions)	76 against 33 (the best single published rule [53]) of 200 held substrates; reconstruction	a nearest-neighbour method 75; the other-family control is not an independent test
ARC-AGI-3 world models: 12 held public games [32]	1,204 against 902 (fixed language) and 1,125 (another game's failure ranking) of 3,168 frames; copying the previous frame 930	probe results, not a competition score; pre-registered test not passed (interval over games 0.0 to 0.232); gains in 3 of 12 games

Table 12: The other held tests of the rebuilt loop, each through its own adapter; Table 14 lists all twelve domain families of this paper (the full paper adds a thirteenth, drug response, a negative). Logic and gene networks are in the text; measured data in Section 8.

The controls in one table. Table 13 gives every control of Sections 4 to 6, one change per row.

what changes (one per row)	correct	what changes (one per row)	correct
construction test: the task's own typed holes (60 admitted, 4 by a fitting fallback)	64 of 95 tasks	held-back halves (exposed): reuse of operations built on the other half (primary)	227 of 624 query outputs
the raw demonstrations as the hole	48	static search in the starting language, same CPU allowance	118
another task's holes (all in the one world whose removed operations compose; 0 of 59 in the other two)	16	reuse with the operation registry shuffled across benchmarks	6
the same 211 operations in random order, without holes	32 (3 in all three repeats)	object-level construction from each task's own demonstrations where reuse failed (secondary)	343
static search in the starting language (8.47 measured CPU-hours; construction charged 2.61, measured about 0.31 plus 902 diagnosis CPU-seconds)	38	the same, with another task's demonstrations	10
learned search order, 500 counted operations per hole (search CPU 115.25 s)	320 of 624 query outputs	a whole-program library instead of operations	37 (static search 56; starting language 6)
hand-set order, same counted operations (106.35 s); at a 40-times larger per-hole cap	234; 329	bounded diagnosis loop on authored Boolean functions, fixed routing	219 of 256 functions
frequency table, or linear model, instead of the learned guide	318, or 323	a fixed build-then-ask schedule (routing used 12.9% less counted work; the schedule 14.8% more; descriptive, not selection evidence)	219

Table 13: Controls, one change per row. Construction counts are tasks solved in at least two of three seeded repeats (64, 48, 16 and 38 also in all three). Held-back halves were held out from each study's development but scored once before by the submitted entries. The 40-times cap is one of four sampled per-hole caps, not total compute and not a proven minimum.

I.2 Motivation and framing

The leaf-lesion motivation. In our unpublished work [49, 50], which does not use CORA, an atlas check flags images outside its reference set, and a pilot found what to change: seeding (where objects start), not geometry; cheap methods, using about 900 times less CPU, matched a geometric solver where it finished (3 of 21 windows). This is motivation only, not CORA evidence.

Neural and symbolic reasoning. Neural systems are strong at the other half of the problem: they propose answers, recognise patterns and spend test-time computation well. The tested neural route predicts output grids directly [65] and gives no program to check, unlike neural systems that write programs [42, 57]. On the 172 inspected public evaluation query outputs, our neural comparison entry, whose checkpoint is contaminated for this set, was correct on 43 in rehearsal and the CORA-only entry on 3 (post hoc, not an official score; Table 9).

Why choosing the change is hard: the left task of Figure 2. Objects that share their size with another object move up; the others move to the bottom border. Five earlier versions of our system failed on it in the same way, with no program that fits both demonstrations, but for four different reasons: the background was assumed black, though one demonstration's is grey; the rule lay beyond the enumerator's reach; every object was bound to one action; and the language had no move by destination. We inspected this task while building CORA, so it illustrates the question; it is not evidence.

I.3 Construction, search and their costs

The object-level constructor of the held-half studies. It reads each grid in 12 views (one-colour or any-colour objects under 4- or 8-connectivity, one object per colour, single cells; background the most common colour or colour 0). Per-object edits are typed expressions over an object: delete; recolour to a constant, to an attribute or through a colour table; shift, move until blocked or to the border, gravitate; mirror, rotate or scale in place; rays in 12 directions; connect to the nearest object; fill enclosed cells or the bounding box; hollow; outline; shifted and mirrored copies. Selection predicates are 24 boolean features (largest, smallest, single cell, has a hole, touches the border and others), value atoms, relations, their negations and conjunctions of two. A candidate operation is one or two first-match rules (predicate, edit) painted over the input, or a whole-output form (crop to the selected object, a token of its colour, a count strip, symmetry completion), kept only when it maps every hole pair exactly (`experiments/object_growth_20261105/REPORT_DEV.md`, section 1).

Construction at scale: blind trial and compute. The blind trial added the 211 operations one at a time in random order under the same checks; it solved 3 tasks in all three repeats, 32 by majority and 74 in at least one, so its majority count is not a stable estimate. Static search used 8.47 CPU-hours of measured kernel CPU on the 95 tasks, counting each search's standing attempt. Construction's three repeats together were charged 2.61 CPU-hours under the protocol's complete-cost rule, which counts cached admission checks at their first computation and the shared diagnosis once per unit; their measured process CPU was about 0.31 CPU-hours, with the shared diagnosis measured separately (902 CPU-seconds). The 3.2 ratio therefore compares measured static CPU with charged construction CPU.

Construction at scale: what was fixed in advance. For 7 of the 11 removed operations' families the space of 211 operations was committed before the operation was chosen, and membership in the space was a condition for choosing it; the other 4 operations had been fixed for an earlier construction test before the space was written. In every family the removed operation is therefore a member of the space by design, which is why the study tests selection within a known space; the blind control searches the same space with the same checks.

The learned order's CPU accounting. At 500 counted operations per hole, the recorded arm-search CPU was 115 against 106 seconds (constructor plus the retries the arm computed first); the constructor CPU alone was 113.78 against 102.61 seconds, and the guide's arms also used 1.04 seconds of shared guide-feature work. Shared setup and reference work are reported separately in the study's audit.

I.4 Transfer: further detail

Logic functions: secondary tests. The advantage held at 12.5% and 50% probes and on 120 synthetic access-control policies generated from real ClassBench seed statistics (all eight pre-registered secondary comparisons rejected after Holm adjustment). On those policies a decision-list learner recovered 49 against the loop's 44, a descriptive reference with no pre-registered test.

Drug response. In the full paper's thirteenth domain family, typed rules revised on development compounds and applied frozen to 416 held-out compounds of the PRISM drug screen gave mean balanced accuracy 0.5249 with CORA's diagnosis against 0.5295 for trying every repair at equal compute: a negative, reported in full in the full paper.

Gene networks: further detail. The regulator replication's documented regulators number 222; in the chosen-observation protocol, a permuted-variable control gave 297 exact rules (Appendix F).

domain family	result	label
two-dimensional grids: ARC-AGI-2, ConceptARC, Mini-ARC	0.83% official; precision 81.5% and 95.0%; 64 against 38 (static search: 3.2x construction's charged CPU) and 16 (another task's holes) of 95; held halves: construction 57 against 30 of 315, reuse 11 against 30	official; generated; held, exposed
one-dimensional sequences: 1D-ARC	precision 95.8%; reuse 216 against 88 of 309	held, exposed
list functions: BIG-bench (construction)	4 of 9 against 1, inconclusive	authored tasks
Boolean gene networks: published models	101 against 66 for each of three controls, one with shuffled regulators, of 595 (all one-input rules); 320 against 291; 433 against 321, a classical learner also 433	published-model reconstruction; reused cohort, first scored by the 101 replication
single-cell perturbations: Perturb-seq	0 of 64 rules admitted; a follow-up admitted none under any tested representation or acceptance rule	real measurements; diagnosis
microscopy: three image sets, then PUMA melanoma	0 strict edits; a tolerant rule's 32 NuInsSeg edits: 10 beneficial, 16 harmful, 6 neutral; a calibrated acceptance rule made 605 edits on 100 fresh regions, 430 harmful	real measurements; diagnosis
spatial transcriptomics: one donor	58 corrections against 65 harms; neighbour guard 36 against 12 (an exploratory ablation; the primary failed its criterion)	real measurements; diagnosis
logic functions: Espresso benchmark files	45 against 7 (two-input gates) and 12 (shuffled failure description) of 217 held functions, 34 to 1; descriptive references CART 39, Espresso-style 38, and on 120 synthetic policies a decision-list learner 49 against the loop's 44; 153 by none of six methods	real benchmark functions; reconstruction
spreadsheet text: SyGuS	22 against 17 (plain search at four times the budget) and 17 (another task's failure) of 53 (46 real forum problems), Holm-adjusted $p = 0.125$ each; plain search at 16 times the budget also 22: mainly faster search	real tasks; reconstruction
cell signalling: the DREAM4 subset of the liver network of Saez-Rodriguez et al. [59], one prior edge removed per task	863 against 731 (static) and 805 (shuffled description; 50 tasks won, 11 lost) of 1,000 held conditions; 44 of the 58 gained over the shuffled control from 1 of 6 removable edges; elsewhere 2.0 points, not significant	synthetic tasks on a real network; one removed prior edge, which the static program cannot represent; replicates a development finding
enzyme chemistry: EnzymeMap reactions [29], 3 examples per family	76 against 33 (the best single published rule [53], chosen on the same 3 examples) of 200 held substrates; a nearest-neighbour method 75; the other-family control is not an independent test	real reactions; reconstruction within families, not discovery or transfer across them
ARC-AGI-3 world models: 12 held public games [32]	1,204 against 902 (fixed language) and 1,125 (another game's failure ranking) of 3,168 frames exact; copying the previous frame 930; pre-registered test not passed (interval over games 0.0 to 0.232); gains in 3 of 12 games	probe results on public games, not a competition score

Table 14: Twelve of the full paper's thirteen domain families, each through its own adapter; the thirteenth, drug response (a screen of drug responses), was a negative and is reported in the full paper. Gene-network results come from separately written biology code, and the last five rows rebuilt the loop. Where it ran, each row has one control, the same loop reading another item's evidence or a shuffled failure description: the loop's own evidence won on generated grid tasks, in logic and in signalling (concentrated in one of six removable edges, with a shuffle weaker than one within each time point), was not significant in spreadsheets, and in the ARC-AGI-3 probe came from one game, so the failure description is not established there; in chemistry the control is not independent, and nearest neighbour recovered 75 against 76 (interval for the difference −0.110 to 0.115), no advantage shown. List functions (inconclusive) and the three measured-data rows, where the initial checks admitted nothing or did net harm, have no such control.